

软件工程三级项目

校园学术文献智能检索与综述生成系统

组号	第 3 组						
组长	学号	202411040239	姓名	周思源	班级	计算机 1 班	40%
组员	学号	202411040232	姓名	黄文政	班级	计算机 1 班	15%
组员	学号	202411040254	姓名	李俊宇	班级	计算机 1 班	15%
组员	学号	202411040237	姓名	郝思怡	班级	计算机 1 班	15%
组员	学号	202411040256	姓名	王淑珍	班级	计算机 1 班	15%

摘要

随着高校课程学习和科研训练中学术资料数量不断增加,学生在查找、筛选、收藏和整理文献时需要更高效的辅助工具。传统关键词检索虽然实现简单,但在自然语言主题理解、跨学科相关性判断、相似文献发现和综述材料整理方面存在一定不足。本文面向软件工程课程三级项目场景,设计并实现了一个校园学术文献智能检索与综述生成系统。系统采用 React + Vite + TypeScript 构建前端,采用 Spring Boot + MyBatis-Plus 构建主后端,采用 MySQL 保存结构化数据,采用 FastAPI + sentence-transformers + FAISS 构建智能语义检索服务,并通过 OpenAI-compatible LLM API 支持在线增强综述生成。

系统面向普通用户提供注册登录、文献普通检索、高级检索、多学科智能语义检索、文献详情查看、收藏与取消收藏、我的收藏、相似文献推荐、智能推荐参考文献、离线综述生成、在线 LLM 增强综述生成以及综述记录查看和筛选等功能。管理员端提供文献管理、分类管理、用户管理、真实数据统计、智能索引重建和 LLM API 配置管理等功能。智能检索不是简单 FAISS 检索,而是在 backend-ai 语义召回基础上,由 Spring Boot 扩大召回候选池,结合多学科主题画像、Query Expansion、MySQL 关键词补充召回、keywordScore、categoryScore、weakPenalty 和 finalScore 进行综合重排,并通过 matchReason 解释推荐原因。

综述生成流程借鉴 RAG (Retrieval-Augmented Generation) 思想,先通过智能检索获得主题相关文献,再由用户选择 2 到 5 篇文献作为参考材料,后端基于选中文献的标题、关键词、摘要和正文节选组织生成上下文。离线 rule 模式基于本地规则进行结构化归纳,是系统默认稳定模式;在线 llm 模式调用管理员配置的大模型 API 进行增强表达,是可选增强模式;当在线 LLM 不可用、超时或输出不完整时,系统自动降级为 llm_fallback_rule。系统对 LLM 输出进行完整性校验,后端强制补全或替换参考文献来源,并清理非法引用编号,避免模型编造不存在的参考文献。

关键词: 学术文献检索;多学科语义检索;综述生成;RAG; FAISS; LLM API; Spring Boot

Abstract

With the increasing amount of academic materials in university courses and research training, students need more efficient tools for searching, filtering, collecting, and organizing literature. Traditional keyword retrieval is easy to implement, but it has limitations in natural language understanding, cross-disciplinary relevance judgment, similar literature discovery, and review material organization. This project designs and implements a campus academic literature intelligent retrieval and review generation system for a software engineering course project. The frontend is built with React, Vite, and TypeScript; the main backend is built with Spring Boot and MyBatis-Plus; MySQL stores structured data; and FastAPI, sentence-transformers, and FAISS provide the intelligent semantic retrieval service. The system also supports online enhanced review generation through an OpenAI-compatible LLM API.

For ordinary users, the system supports registration and login, normal retrieval, advanced retrieval, multi-disciplinary semantic retrieval, literature detail viewing, favorites, similar literature recommendation, intelligent reference recommendation, offline review generation, online LLM-enhanced review generation, and review history filtering. For administrators, the system supports literature management, category management, user management, real-data statistics, intelligent index rebuilding, and LLM API configuration management. The intelligent retrieval module is not a simple FAISS retrieval process. Based on semantic recall from backend-ai, Spring Boot expands the candidate pool, identifies multi-disciplinary topic profiles, performs query expansion, supplements recall through MySQL keyword matching, and re-ranks candidates with keywordScore, categoryScore, weakPenalty, and finalScore, while returning matchReason for recommendation explanation.

The review generation process follows a RAG-style approach. The system first retrieves topic-related literature, then the user selects two to five papers as reference materials, and the backend organizes the selected literature's titles, keywords, abstracts, and content excerpts as the generation context. The offline rule mode is the stable default mode based on local rules, while the online llm mode is an optional enhancement that calls the administrator-configured LLM API. When the online LLM is unavailable, times out, or produces incomplete output, the system falls back to llm_fallback_rule. The backend validates the generated content, completes or replaces reference sources, and removes invalid citation numbers to prevent fabricated references.

Keywords: academic literature retrieval; multi-disciplinary semantic retrieval; review generation; RAG; FAISS; LLM API; Spring Boot

目录

摘要	I
Abstract	II
1 引言	1
1.1 项目背景	1
1.1.1 校园学术文献检索与综述写作需求	1
1.1.2 传统检索和自由生成方式的不足	1
1.1.3 大模型增强与 RAG 思想的引入	1
1.2 项目目标	2
1.2.1 普通用户端目标	2
1.2.2 管理员端目标	2
1.2.3 软件工程实践目标	2
1.3 参考资料	3
2 可行性研究	4
2.1 技术可行性	4
2.1.1 前端技术可行性	4
2.1.2 后端与数据库技术可行性	4
2.1.3 智能检索与 LLM 接入可行性	4
2.2 经济可行性	5
2.2.1 开源框架与本地运行成本	5
2.2.2 DeepSeek API 可选调用成本	5
2.3 操作可行性	5
2.3.1 普通用户操作可行性	5
2.3.2 管理员操作可行性	6
2.4 可行性结论	6
2.5 竞品分析	6
2.5.1 与学术检索平台对比	6
2.5.2 竞品分析结论	7
3 需求分析	8
3.1 功能需求	8
3.1.1 功能需求总体结构	8
3.1.2 普通用户功能结构化说明	9
3.1.3 管理员功能结构化说明	10
3.1.4 综述生成材料约束需求	10
3.2 数据流图 DFD	11
3.2.1 顶层数据流图	11
3.2.2 0 层数据流图	12
3.2.3 智能检索 1 层数据流图	13
3.2.4 综述生成 1 层数据流图	14
3.3 数据字典	14
3.3.1 数据项	14
3.3.2 数据流	15
3.3.3 数据存储	15

3.3.4 数据处理	16
3.4 非功能需求	16
3.4.1 可用性和可靠性	16
3.4.2 安全性和权限控制	17
3.4.3 可维护性和扩展性	17
4 系统设计（总体设计）	18
4.1 系统模块划分	18
4.1.1 总体架构	18
4.1.2 功能结构	19
4.2 模块结构图（SC 图）	19
4.3 接口设计	20
4.3.1 REST API 设计原则	20
4.3.2 主要业务接口	20
4.3.3 backend-ai 接口	22
4.4 数据库设计	23
4.4.1 E-R 图	23
4.4.2 核心表结构	23
4.4.3 两级分类与模拟数据	24
5 详细设计	25
5.1 模块算法设计	25
5.1.1 普通检索与高级检索算法	25
5.1.2 智能检索重排算法	26
5.1.3 backend-ai 向量检索算法	27
5.1.4 综述生成算法	28
5.1.5 LLM 输出校验与降级算法	30
5.1.6 面向对象类图	31
5.2 输入/输出设计：界面原型设计	32
5.2.1 用户输入设计	32
5.2.2 用户输出设计	32
5.2.3 界面原型与运行效果	33
5.3 代码规范	33
5.3.1 后端代码规范	33
5.3.2 前端代码规范	33
5.3.3 文档与安全规范	33
6 编码实现	34
6.1 开发环境	34
6.2 核心代码片段	34
6.2.1 普通检索和高级检索代码	34
6.2.2 智能检索重排代码	35
6.2.3 backend-ai FAISS 索引代码	35
6.2.4 综述生成双模式代码	36
6.2.5 LLM 输出校验代码	37
6.2.6 API Key 脱敏代码	37
6.2.7 JWT 权限拦截代码	38

6.3 代码结构说明	39
7 测试	40
7.1 测试计划	40
7.2 测试用例	40
7.3 缺陷分析	41
8 维护计划	43
8.1 纠错性维护	43
8.2 适应性维护	43
8.3 完善性维护	43
9 用户手册	44
9.1 安装说明	44
9.1.1 数据库准备	44
9.1.2 backend-ai 启动	44
9.1.3 backend-java 启动	44
9.1.4 frontend 启动	44
9.2 使用指南	45
9.2.1 普通用户使用流程	45
9.2.2 管理员使用流程	52
10 项目总结	57
10.1 成果总结	57
10.2 经验与不足	57
10.3 小组成员分工及自评	58

1 引言

1.1 项目背景

1.1.1 校园学术文献检索与综述写作需求

高校学生在课程论文、课程设计、科研训练和毕业设计准备过程中，经常需要围绕一个研究主题查找多篇学术资料，再对文献进行筛选、比较、归纳和综述写作。传统做法通常依赖搜索引擎、图书馆检索系统、知网或万方等平台，再由学生手工整理资料。对于初学者而言，文献标题、关键词、摘要和研究方向之间存在理解门槛，尤其在跨学科场景中，同一关键词可能对应不同语境。例如“人工智能在教学中的应用”和“人工智能在医学影像中的应用”都包含“人工智能”，但前者侧重教育技术、学习行为和教学评价，后者侧重临床诊断、医学影像和健康管理。

本项目面向校园课程项目场景，构建一个可运行、可演示、可维护的模拟文献检索与综述生成系统。系统不是替代真实论文数据库，也不声称拥有真实论文资源，而是通过模拟文献数据和完整业务流程展示软件工程开发过程。项目中使用的 500 条文献来自课程项目模拟数据脚本，覆盖自然科学、工程技术、医学健康、农业生命、社会科学、人文艺术等方向，用于支撑普通检索、语义检索、相似推荐和综述生成演示。

1.1.2 传统检索和自由生成方式的不足

传统关键词检索主要依赖输入词与文献字段之间的字面匹配。它适合用户已经掌握明确主题词、作者、年份或 DOI 的场景，但对自然语言查询、同义表达和跨学科语义关系支持不足。例如用户输入“大学生心理健康智能预警”时，字面检索可能遗漏包含“心理危机干预”“情绪识别”“学生画像”等表达的文献；而只使用向量语义召回又容易受到宽泛词汇影响，导致“人工智能”相关但学科方向不一致的文献混入结果。

综述生成同样存在两个极端问题：完全人工整理耗时较长，直接让大模型自由生成又容易出现参考文献来源不清、引用编号不一致或编造不存在文献的问题。课程项目需要展示一个可控的信息系统，而不是只调用一个外部问答模型。因此，本系统将检索、参考文献选择、生成上下文组织、离线规则生成、在线 LLM 增强、输出校验和记录保存串联起来，使综述生成过程有依据、有边界、可回溯。

1.1.3 大模型增强与 RAG 思想的引入

RAG 的核心思想是先检索相关材料，再基于检索结果进行生成，从而增强生成内容的依据性和可追溯性。本项目没有实现复杂 chunk 管理、专门向量数据库、多轮 Agent 或工业级引用溯源系统，因此不把系统描述为完整工业级

RAG 平台，而是采用 RAG 思想和 RAG 风格综述生成流程。具体表现为：用户输入综述主题后，系统先推荐相关文献；用户选择 2 到 5 篇文献；后端提取选中文献的标题、关键词、摘要和正文节选；rule 模式基于本地规则归纳生成；llm 模式将受约束材料传给管理员配置的 LLM API；后端校验生成结果并补全参考文献来源；最后保存 review_record 和 generationMode。

这种设计既利用了语义检索和 LLM 的能力，也保留了课程项目所需的稳定性。离线规则生成是默认稳定模式，在线 LLM 是增强模式，不是系统唯一生成方式。

1.2 项目目标

1.2.1 普通用户端目标

普通用户端目标是形成从“检索资料”到“整理综述”的完整学习流程。用户能够注册并登录系统，使用普通检索按关键词查找文献，使用高级检索按标题、作者、期刊、DOI、分类、文献类型和年份范围筛选文献，使用智能检索输入自然语言主题并获得带有 similarity、finalScore 和 matchReason 的结果。用户进入文献详情页后可以查看摘要、关键词、正文节选等信息，收藏或取消收藏文献，并通过相似文献推荐继续发现相关资料。

在综述生成环节，用户输入主题后，系统智能推荐参考文献，用户选择 2 到 5 篇文献并选择生成模式。离线 rule 模式不依赖外部 API，保证稳定可用；在线 llm 模式在管理员配置可用时调用大模型增强语言表达；若在线生成失败则自动降级为 llm_fallback_rule。综述记录页面展示历史记录，并支持根据 generationMode 筛选。

1.2.2 管理员端目标

管理员端目标是保证系统数据、配置和智能检索索引可维护。管理员可以维护文献、分类和用户，查看真实数据统计面板，触发智能检索索引重建，并管理 LLM API 配置。文献新增、编辑或删除后，MySQL 中的结构化数据会变化，但 FAISS 索引文件不会自动实时同步，因此系统提供“重建智能索引”入口，由管理员主动触发 backend-ai 重新向量化文献并保存 faiss.index 和 id_map.json。

LLM API 管理目标是让在线增强生成不与具体模型写死绑定。管理员可以新增、编辑、删除配置，设置 active 配置，测试连接，并在前端只看到脱敏后的 API Key。课程演示环境中可以保存配置，生产环境则应进一步使用加密存储、密钥轮换和访问审计。

1.2.3 软件工程实践目标

本项目不仅关注功能实现，还强调软件工程三级项目要求：需求分析要有功能需求、DFD 和数据字典；系统设计要有架构图、功能结构图、SC 图、接口设计、数据库设计和 E-R 图；详细设计要有算法流程、伪代码、输入输出设计、界面原型和代码规范；编码实现要能展示真实代码结构和核心代码片段；测试部分要覆盖计划、用例、边界条件和缺陷分析；最终还要包含维护计划、用户手册、

小组分工和 AI 辅助记录。

1.3 参考资料

项目设计与实现参考了软件工程课程教材中关于可行性研究、需求分析、数据流图、数据字典、总体设计、详细设计、测试和维护的基本方法；参考了 Spring Boot、React、Vite、MyBatis-Plus、MySQL、FastAPI、FAISS、sentence-transformers 等官方文档；参考了 RAG、向量检索和 LLM 工程化输出约束相关资料；在线增强生成的费用分析参考了 DeepSeek 官方 API 价格页。系统中涉及的账号、密钥和数据库连接信息在报告中均使用描述性文字或占位说明，不展示真实敏感信息。

2 可行性研究

2.1 技术可行性

2.1.1 前端技术可行性

前端采用 React + Vite + TypeScript。React 适合构建组件化页面，Vite 支持快速开发和构建，TypeScript 能够通过类型约束降低接口字段使用错误。项目中前端页面覆盖首页、登录注册、搜索页、文献详情页、收藏页、综述生成页、综述记录页、管理员后台、文献管理、分类管理、用户管理、统计面板和 LLM 配置管理页面。前端通过 `src/api` 目录封装请求，使用统一请求基地址访问 Spring Boot 的 REST API，便于接口变更时集中调整。

从课程项目角度看，前端技术成熟、生态完整、开发成本可控。系统界面已经通过截图保存运行结果，能够支撑用户手册和答辩演示。

2.1.2 后端与数据库技术可行性

主后端采用 Spring Boot 3.2 + MyBatis-Plus + MySQL。Spring Boot 负责 REST 接口、业务逻辑、权限控制、异常处理和服务间调用；MyBatis-Plus 负责实体映射、分页查询和 CRUD；MySQL 存储用户、分类、文献、收藏、检索历史、综述记录、LLM 配置等结构化数据。项目使用 Java 17，依赖清晰，适合课程项目开发和演示。

数据库表结构已由 SQL 脚本明确给出，主要通过逻辑关联维护关系。虽然脚本中未显式声明外键，但字段关系清楚，例如 `favorite.user_id` 关联 `user.id`，`favorite.literature_id` 关联 `literature.id`，`literature.category_id` 关联 `category.id`，`category.parent_id` 形成两级分类，`review_record.user_id` 关联 `user.id`。这种设计降低了课程演示中导入模拟数据和调整脚本的复杂度。

2.1.3 智能检索与 LLM 接入可行性

智能检索服务采用 FastAPI + sentence-transformers + FAISS。backend-ai 启动后加载多语言 sentence-transformers 模型，将文献标题、分类、文献类型、关键词、摘要和正文节选拼接为向量化文本，建立 FAISS IndexFlatIP 索引，并把 FAISS 位置到文献 ID 的映射保存为 `id_map.json`。Spring Boot 通过 HTTP 调用 backend-ai 的 `/semantic-search` 和 `/recommend` 接口，再结合数据库补充召回和业务重排。

在线 LLM 增强生成通过 OpenAI-compatible Chat Completions API 实现，能够适配 DeepSeek 等兼容接口。系统把 Base URL、Model、API Key、Timeout 放入管理员可配置项，避免把模型服务写死在代码中。由于 LLM 可能超时、返回错误或输出不完整，系统通过 rule 离线模式兜底，技术风险可控。

2.2 经济可行性

2.2.1 开源框架与本地运行成本

项目主体使用开源框架和本地服务，React、Vite、Spring Boot、MyBatis-Plus、MySQL、FastAPI、sentence-transformers 和 FAISS 均可在课程项目环境中免费使用。开发和演示主要依赖学生已有电脑，运行成本主要是本地 CPU、内存和磁盘资源。500 条模拟文献规模较小，FAISS 索引文件体积可控，不需要购买云服务器或专门向量数据库服务。

表 2-1 本地运行成本分析

成本项	说明	课程项目成本判断
前端开发与运行	Vite 本地开发服务器	免费，主要消耗本机资源
Spring Boot 后端	Java 17 + Maven 本地运行	免费，依赖本机环境
MySQL 数据库	本地 MySQL 8.x	免费或已由课程环境提供
backend-ai	FastAPI + sentence-transformers + FAISS	免费，首次模型下载耗时较长
文献数据	课程项目模拟数据	无真实论文购买成本

2.2.2 DeepSeek API 可选调用成本

在线 LLM 不是系统唯一生成方式，用户选择 rule 模式时不产生外部 API 调用费用。若管理员配置 DeepSeek API 并启用在线 llm 模式，费用按照 DeepSeek 官方 API 价格页以 token 计费。以 2026 年 5 月 24 日查询的 DeepSeek 官方价格页为依据，deepseek-chat 对应 deepseek-v4-flash 非思考模式，输入缓存命中价格为 0.028 美元/百万 tokens，输入缓存未命中价格为 0.28 美元/百万 tokens，输出价格为 0.42 美元/百万 tokens；deepseek-reasoner 对应 deepseek-r1-0528 思考模式，输入缓存命中价格为 0.14 美元/百万 tokens，输入缓存未命中价格为 0.55 美元/百万 tokens，输出价格为 2.19 美元/百万 tokens。实际课程演示只在少量在线综述生成或测试连接时调用 API，因此总体成本较低且可控。

本项目仍把离线规则生成作为默认稳定模式，原因是课程验收不应依赖第三方 API 账户余额、网络连通性或外部服务状态。在线 LLM 的价值在于增强表达质量和演示工程扩展能力，而不是替代本地业务逻辑。

2.3 操作可行性

2.3.1 普通用户操作可行性

普通用户操作流程接近常见信息检索系统：先登录，再搜索文献，查看详情，

收藏文献，选择参考文献，生成综述，查看记录。普通检索适合明确关键词，高级检索适合精确筛选，智能检索适合自然语言主题探索。综述生成页面将“推荐参考文献”“选择 2 到 5 篇”“选择生成模式”“查看生成结果”组织在同一流程中，减少用户在多个工具之间切换的负担。

2.3.2 管理员操作可行性

管理员操作集中在后台。文献管理、分类管理、用户管理和统计面板对应常见后台管理习惯；索引重建入口放在管理员端，避免普通用户误触发耗时操作；LLM API 管理页面提供新增、编辑、删除、设为 active 和测试连接功能。API Key 在列表中脱敏展示，避免截图或演示时暴露完整密钥。整体操作符合课程项目演示和后续维护需要。

2.4 可行性结论

从技术方面看，前端、后端、数据库、智能检索服务和在线 LLM 接口均采用成熟技术栈，项目规模适合课程三级项目。从经济方面看，系统主体依赖开源框架和本地运行，在线 LLM 为可选增强模式，调用成本可控。从操作方面看，普通用户和管理员流程清晰，截图和演示路径完整。从风险控制方面看，系统通过离线 rule 模式、llm_fallback_rule 降级、API Key 脱敏和管理员权限控制降低运行风险。因此，本项目具备可行性。

2.5 竞品分析

2.5.1 与学术检索平台对比

知网、万方等平台拥有真实论文资源、版权体系和成熟检索能力，是专业学术资源平台。本项目无法也不需要资源规模上与其竞争。本项目的价值在于面向课程实践，把“模拟文献库、语义检索、可解释重排、受约束综述生成和后台维护”整合成一个可运行系统。通用大模型工具具备较强文本生成能力，但如果用户直接输入主题让模型生成综述，容易出现来源不可控和参考文献虚构问题；本项目通过用户选择文献、后端组织上下文和参考文献补全来约束生成来源。

表 2-2 同类系统或工具对比

对比对象	主要优势	与本项目差异	本项目定位
知网、万方等论文平台	真实论文资源丰富，检索字段完善	项目不具备真实论文版权库和资源规模	用模拟数据展示软件工程实现流程
高校图书馆检索系统	与学校资源和馆藏绑定	通常侧重资源检索，课程项目难以接入真实馆藏	构建可演示的本地文献检索闭环
通用大模型问答工具	语言生成能力强，交互灵活	参考来源不一定绑定本地选中文献	只把 LLM 作为受约束增强模式
本项目	检索、推荐、综述、后台维护整合	数据为课程模拟数据，不是生产级平台	教学型、可解释、可演示的检索增强式文献系统

2.5.2 竞品分析结论

本项目的差异化定位不是资源规模，而是工程闭环。系统明确实现了 MySQL 文献管理、FAISS 语义索引、Spring Boot 多学科重排、RAG 风格综述生成、LLM 输出校验、参考文献后端补全、generationMode 记录和管理员 LLM 配置管理。相比单纯页面展示或单纯模型问答，本项目更能体现软件工程课程要求中的需求分析、设计、实现、测试、维护和文档整理过程。

3 需求分析

3.1 功能需求

3.1.1 功能需求总体结构

系统主要角色包括普通用户、管理员、backend-ai 智能检索服务和在线 LLM API。普通用户关注文献查找、详情阅读、收藏、推荐和综述生成；管理员关注文献、分类、用户、统计、索引和 LLM 配置维护；backend-ai 负责向量化、索引重建、语义召回和相似推荐；在线 LLM API 仅在用户选择在线增强生成时被调用。

图 5-1 普通用户用例图

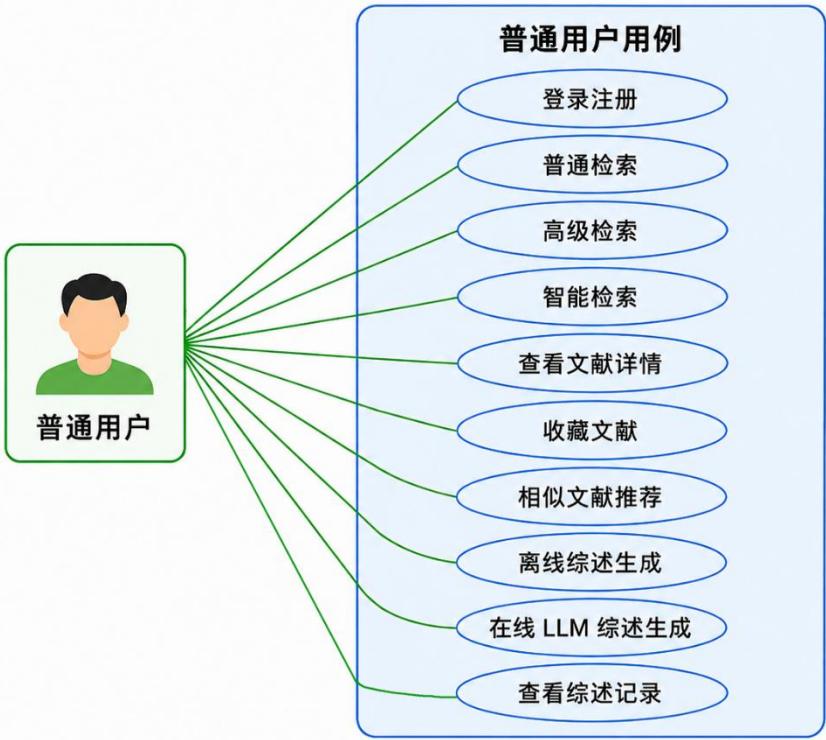


图 5-1 普通用户用例图

普通用户用例图体现了用户从注册登录到检索、收藏、推荐、综述生成和记录查看的完整流程。该图说明普通用户不是只进行单次搜索，而是围绕学习写作任务完成持续的资料整理。

图 5-2 管理员用例图

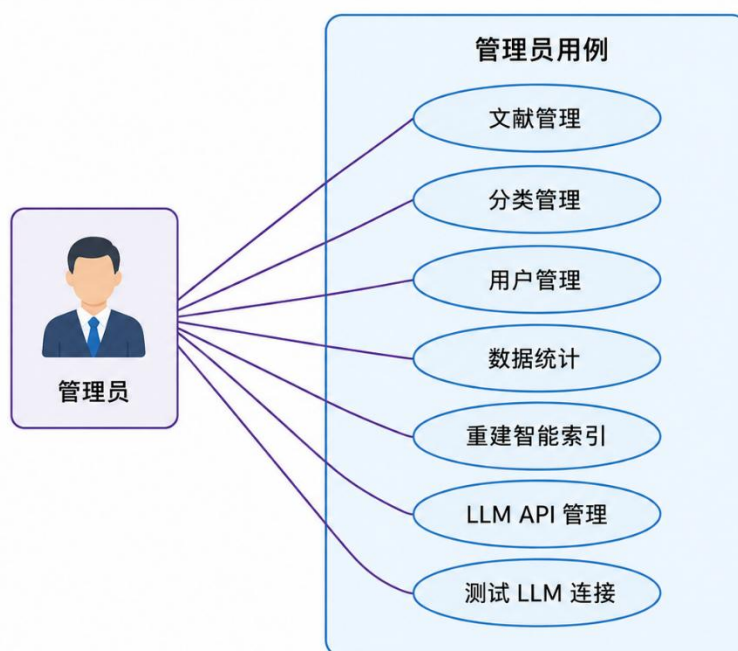


图 5-2 管理员用例图

管理员用例图体现了后台维护能力。文献更新后需要重建智能索引，LLM API 配置影响在线综述生成，因此这些功能必须放在管理员端并进行权限控制。

3.1.2 普通用户功能结构化说明

表 3-1 普通用户功能需求结构化说明

功能	输入	处理	输出
注册登录	用户名、密码、邮箱等	校验参数，密码 BCrypt 存储，登录成功签发 Token	用户信息、Token 或错误提示
普通检索	keyword、分页、排序	匹配标题、摘要、关键词、正文节选	文献列表、总数、分页结果
高级检索	标题、作者、期刊、DOI、分类、年份范围、类型	根据非空条件组合查询	符合条件的文献列表
智能检索	自然语言 query、topK	backend-ai 语义召回，Spring Boot 主题画像与重排	带 similarity、finalScore、matchReason 的结果
文献详情	文献 ID	查询文献元数据和正文节选	文献详情、相似推荐入口
收藏管理	用户 ID、文献 ID	新增或删除收藏，防止重复收藏	收藏状态或收藏列表
综	主题、2 到 5	rule 本地生成	综述正文、参考

述生成	篇文献、mode	或 llm 在线增强，失败时降级	来源、generationMode
综述记录	当前用户、筛选条件	查询 review_record 并按模式筛选	历史综述列表

3.1.3 管理员功能结构化说明

表 3-2 管理员功能需求结构化说明

功能	输入	处理	输出
文献管理	文献标题、作者、摘要、关键词、分类等	新增、编辑、删除、查询 MySQL 文献	文献列表或操作结果
分类管理	分类名称、父分类、排序	维护两级分类体系	分类树或分类列表
用户管理	用户状态	禁用或启用用户	用户列表和状态
数据统计	管理员请求	聚合用户、文献、收藏、综述、检索、LLM 配置等真实数据	统计卡片和图表数据
重建智能索引	管理员操作	读取 MySQL 文献并调用 backend-ai/rebuild-index	索引重建结果和数量
LLM API 管理	Base URL、Model、API Key、Timeout	新增、编辑、删除、设为 active、测试连接	脱敏配置列表和测试结果

3.1.4 综述生成材料约束需求

综述生成必须基于用户选择的文献材料，不能让 LLM 自由编造参考文献。系统规定每次生成需选择 2 到 5 篇文献，后端根据文献 ID 查询实际文献，提取标题、作者、年份、期刊、关键词、摘要和正文节选，再组织为生成上下文。生成结果保存到 review_record，其中 generation_mode 记录 rule、llm 或 llm_fallback_rule。这一需求保证综述内容与本地模拟文献库建立对应关系。

3.2 数据流图 DFD

3.2.1 顶层数据流图

图 3-1 顶层数据流图



图 3-1 顶层数据流图

顶层 DFD 将系统视为一个整体。普通用户向系统提交登录请求、检索条件、收藏请求和综述生成请求；管理员提交文献维护、分类维护、用户管理、索引重建和 LLM 配置请求；系统与 MySQL 交换结构化数据，与 FAISS 索引文件交换向量检索数据，并在在线增强模式下调用外部 LLM API。

3.2.2 0 层数据流图

图 3-2 0 层数据流图

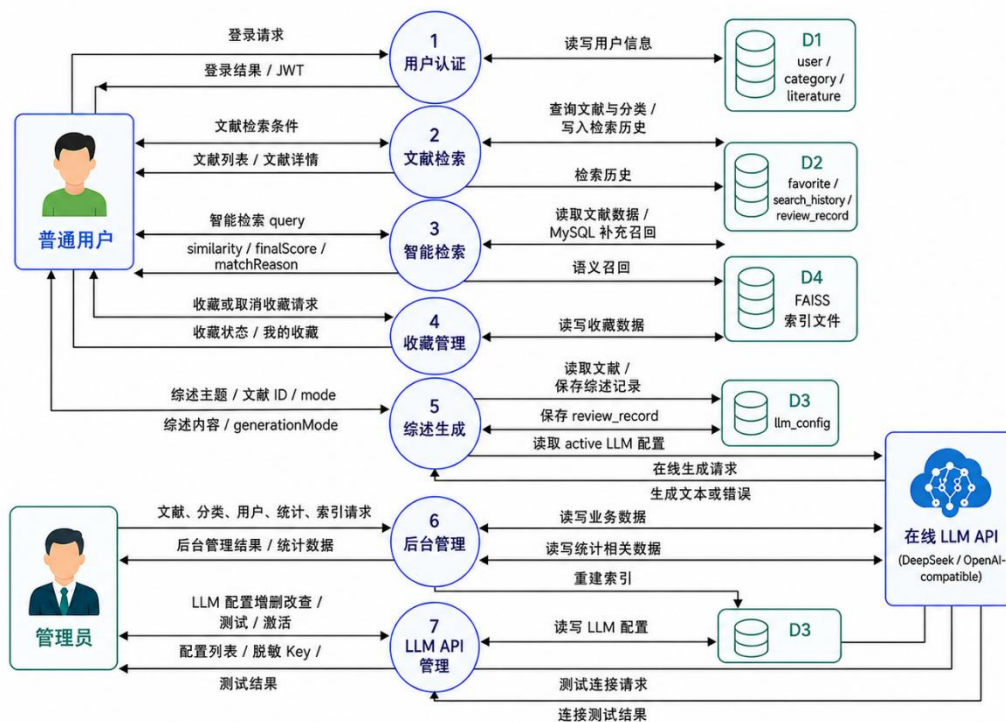


图 3-2 0 层数据流图

0 层 DFD 将系统分解为用户认证、文献检索、智能检索、收藏管理、综述生成、后台管理和 LLM API 管理等处理过程。普通检索和高级检索主要访问 MySQL；智能检索同时访问 backend-ai 和 MySQL；综述生成读取文献材料和 active LLM 配置，并写入综述记录；后台管理维护文献、分类、用户和配置。

3.2.3 智能检索 1 层数据流图

图 3-3 智能检索 1 层数据流图

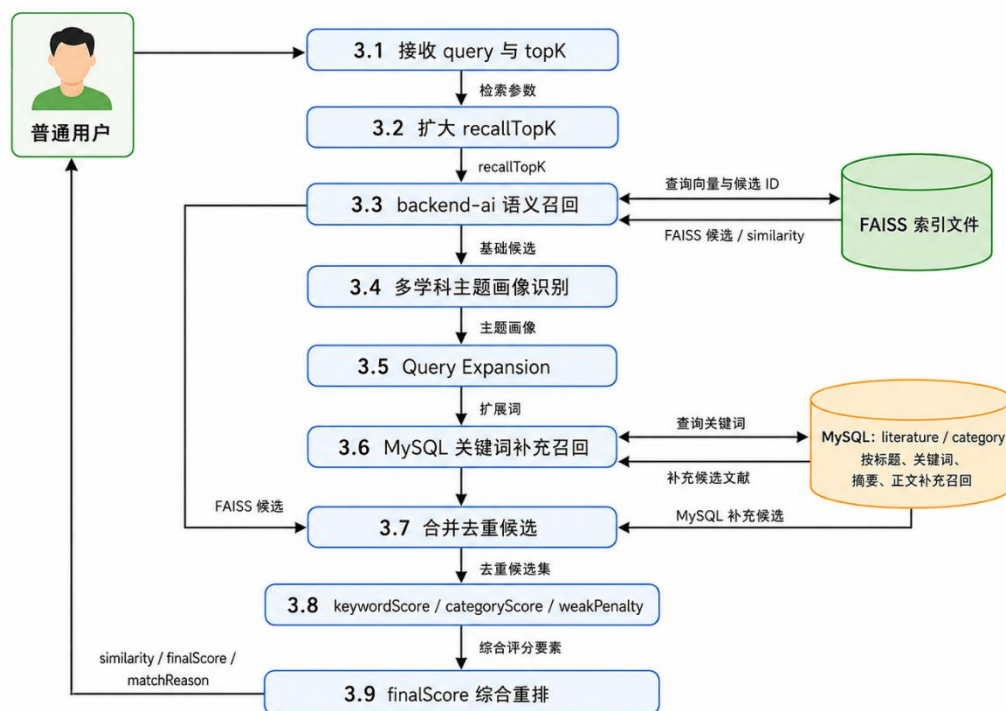


图 3-3 智能检索 1 层数据流图

智能检索 1 层 DFD 展开了 query 到结果的内部数据流。Spring Boot 接收用户 query 后扩大 recallTopK，调用 backend-ai 获取 FAISS 语义召回结果；同时识别多学科主题画像，执行 Query Expansion，并通过 MySQL 关键词补充召回；之后对候选文献合并去重，计算 keywordScore、categoryScore 和 weakPenalty，得到 finalScore 并返回 matchReason。

3.2.4 综述生成 1 层数据流程图

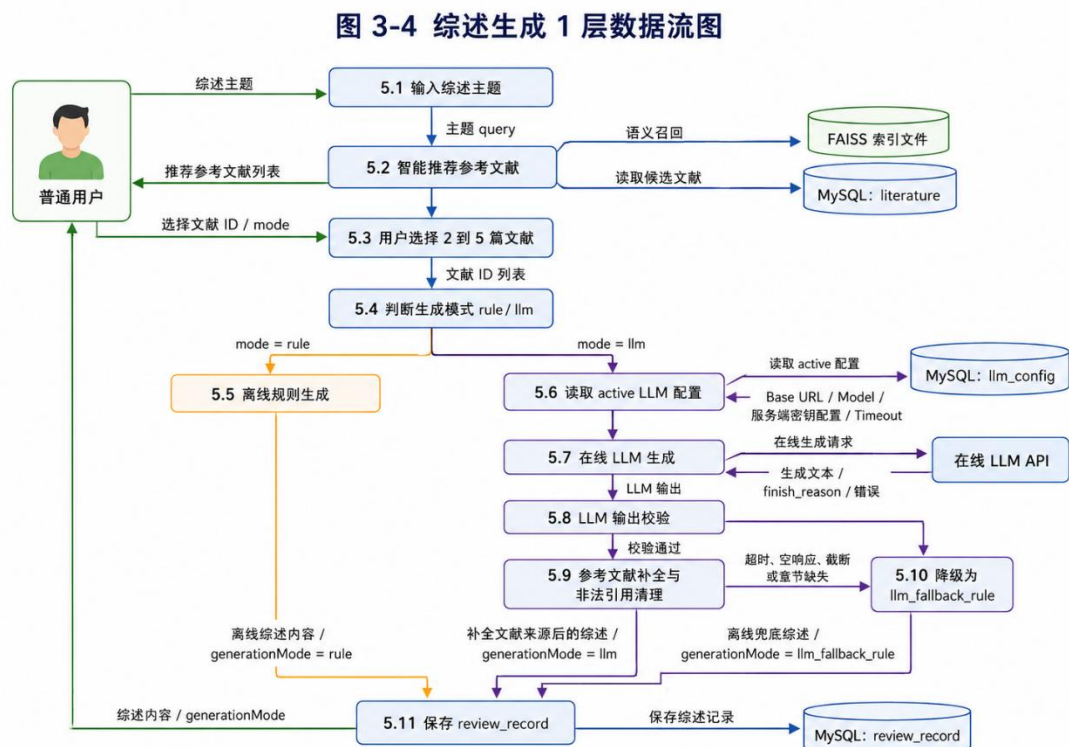


图 3-4 综述生成 1 层数据流程图

综述生成 1 层 DFD 展开了 RAG 风格流程。用户输入主题后，系统先推荐参考文献，用户选择 2 到 5 篇文献后选择 rule 或 llm 模式。rule 模式直接基于本地规则生成；llm 模式读取 active LLM 配置并调用外部 API，输出经过完整性校验、参考文献补全和非法引用清理。无论哪种模式，最终结果都写入 review_record。

3.3 数据字典

3.3.1 数据项

表 3-3 主要数据项字典

数据项	含义	类型或取值	约束说明
用户编号	用户唯一标识	BIGINT	主键，自增
用户名	登录账号	VARCHAR(50)	唯一、必填
角色	用户权限类型	USER / ADMIN	决定是否可访问后台接口
文献编号	文献唯一标识	BIGINT	主键，自增
文献标题	文献名称	VARCHAR(255)	普通检索和向量化重点字段

摘要	文献摘要	TEXT	用于检索、详情和综述材料
关键词	文献关键词	VARCHAR(255)	用于检索和关键词评分
分类编号	文献所属分类	BIGINT	逻辑关联 category.id
综述编号	综述记录唯一标识	BIGINT	主键，自增
generationMode	综述生成方式	rule / llm / llm_fallback_rule	用于记录和筛选
LLM 配置编号	大模型配置标识	BIGINT	active 配置用于在线生成

3.3.2 数据流

表 3-4 主要数据流字典

数据流	来源	去向	主要内容
登录请求	用户或管理员	用户认证模块	用户名、密码
文献检索条件	普通用户	文献检索模块	keyword、分类、年份、排序、分页
智能检索请求	普通用户	智能检索模块	query、topK
智能检索结果	智能检索模块	普通用户	文献列表、similarity、finalScore、matchReason
收藏请求	普通用户	收藏管理模块	用户 ID、文献 ID
综述生成请求	普通用户	综述生成模块	topic、literatureIds、mode
LLM 配置信息	管理员	LLM API 管理模块	provider、baseUrl、model、apiKey、timeout
索引重建请求	管理员	backend-ai	文献向量化输入列表

3.3.3 数据存储

表 3-5 数据存储字典

数据存储	说明	主要字段
user	用户与管理员账号	id、username、password_hash、role、status
category	两级学科分类	id、name、parent_id、sort_order
literature	文献元数据和正文节选	title、authors、abstract_text、keywords、category_id、content
favorite	用户收藏关系	user_id、literature_id

search_history	检索历史	user_id、keyword、search_type、result_count
review_record	综述记录	user_id、topic、literature_ids、content、generation_mode
llm_config	LLM API 配置	provider、base_url、model、api_key、active、timeout_seconds
FAISS 索引文件	backend-ai 本地向量索引	backend-ai/data/faiss.index、id_map.json

3.3.4 数据处理

表 3-6 数据处理字典

处理过程	输入	处理说明	输出
用户认证	登录请求	校验密码、解析或生成 JWT、写入用户上下文	Token、用户信息
普通检索	keyword	匹配标题、摘要、关键词和正文节选	文献列表
高级检索	多字段条件	动态组合查询条件和排序	文献列表
智能检索	query	语义召回、主题识别、扩展召回、综合重排	带解释结果
相似推荐	literatureId	根据目标文献向量检索 topK+1 并排除自身	推荐文献
综述生成	topic、文献 ID、mode	rule 或 llm 生成，失败降级	综述记录
LLM 输出校验	LLM 返回内容	检查 finish_reason、章节、截断、引用编号	合格内容或降级
索引重建	文献列表	向量化并写入 FAISS 文件	索引文件与 ID 映射

3.4 非功能需求

3.4.1 可用性和可靠性

系统需要保证核心流程可用。backend-ai 未启动或索引未重建时，系统应给出明确提示，不影响普通检索、详情和收藏等基础功能。在线 LLM 不可用时，综述生成不应失败中断，而应自动降级为离线规则生成，并记录 generationMode=llm_fallback_rule。管理员统计页必须来自数据库真实聚合，不使用写死的假趋势数据。

3.4.2 安全性和权限控制

系统使用 JWT 进行登录态识别，公开接口包括注册、登录、文献检索、文献详情、分类列表、分类统计、AI 健康检查、智能检索和相似推荐；收藏、综述记录和后台接口需要登录；文献管理、分类修改、用户管理、统计、索引重建和 LLM API 管理需要管理员角色。用户密码使用 BCrypt 哈希保存。LLM API Key 不在前端明文展示，只返回脱敏字段 `apiKeyMasked`。生产环境应进一步使用环境变量、加密存储和密钥轮换。

3.4.3 可维护性和扩展性

系统采用前后端分离和服务分层结构。前端按页面和 API 封装组织，Spring Boot 按 `controller`、`service`、`mapper`、`entity`、`dto`、`vo`、`security` 分层，`backend-ai` 独立提供语义检索服务。MySQL 和 FAISS 分别负责结构化数据与向量索引，职责清晰。后续如果扩展 PDF 上传、全文解析、向量数据库、Cross-Encoder rerank 或流式输出，应作为后续迭代功能，而不能写成本项目已实现功能。

4 系统设计（总体设计）

4.1 系统模块划分

4.1.1 总体架构

图 4-1 系统总体架构图

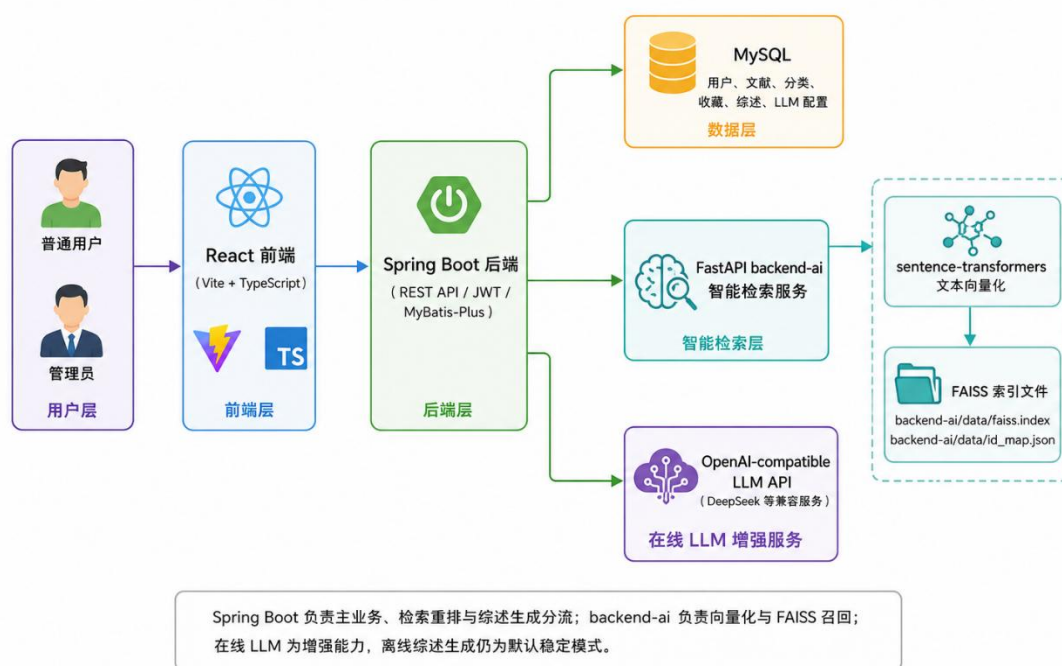


图 4-1 系统总体架构图

系统采用前后端分离和多服务协作架构。React 前端负责页面展示和用户交互；Spring Boot 后端负责认证授权、业务规则、数据库访问、智能检索重排、综述生成控制和管理员功能；MySQL 保存结构化业务数据；backend-ai 使用 FastAPI 对外提供向量索引重建、语义检索和相似推荐；sentence-transformers 负责向量化；FAISS 索引文件保存于 backend-ai/data；DeepSeek / OpenAI-compatible LLM API 作为可选在线增强服务。

4.1.2 功能结构

图 4-2 功能结构图

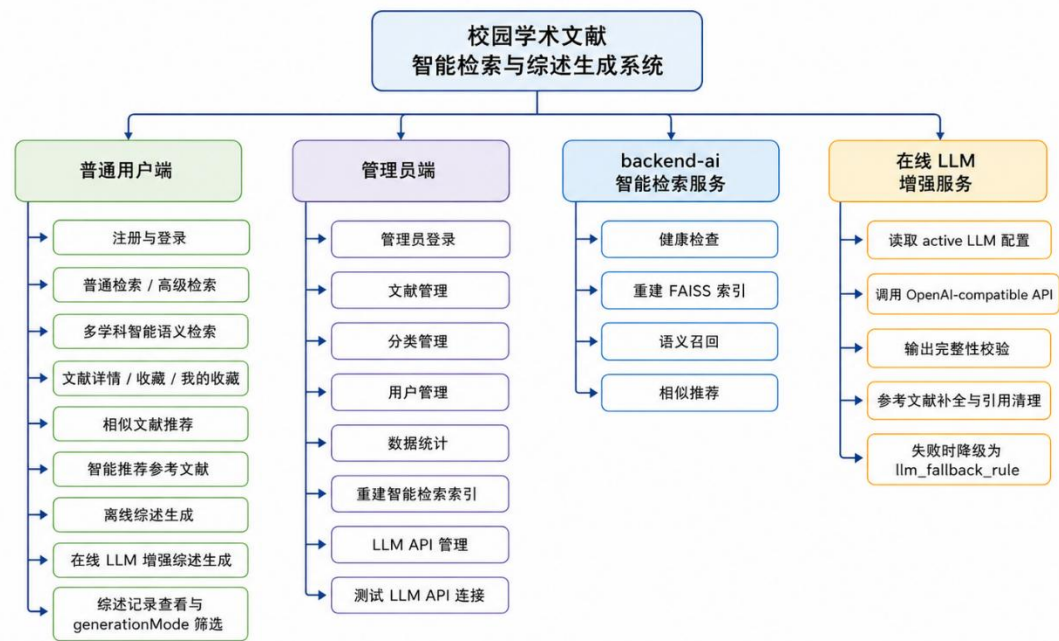


图 4-2 功能结构图

功能结构分为普通用户端、管理员端、backend-ai 智能检索服务和在线 LLM 增强服务。普通用户端围绕文献检索和综述生成展开；管理员端围绕数据维护和配置维护展开；backend-ai 专注语义向量召回；在线 LLM 增强服务只在用户选择在线生成且 active 配置可用时参与。

4.2 模块结构图（SC 图）

图 4-4 模块结构图（SC 图）

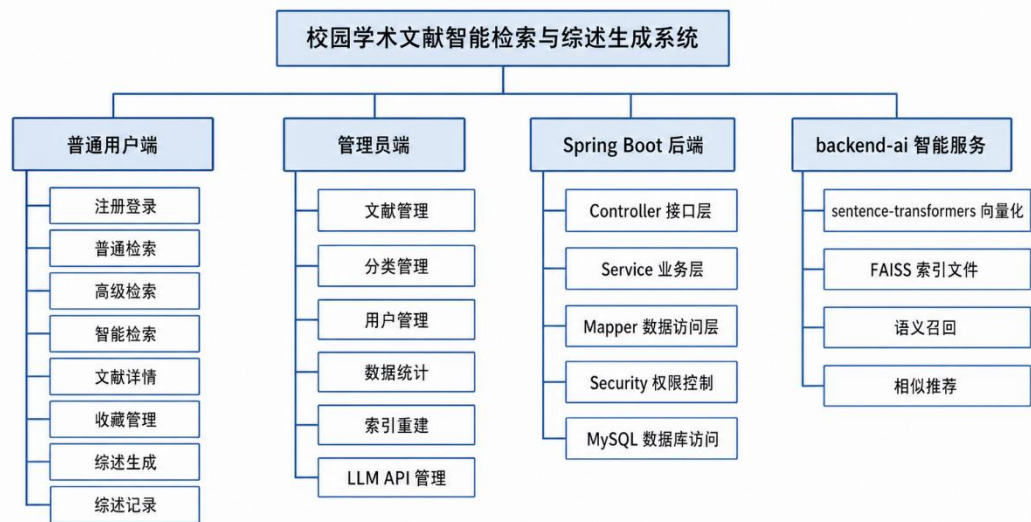


图 4-4 模块结构图（SC 图）

SC 图从模块调用和层次结构角度展示系统。普通用户端和管理员端都通过前端 API 调用 Spring Boot; Spring Boot 的 controller 负责接收请求, service 负责业务处理, mapper 负责数据库访问, security 负责权限上下文。智能检索相关操作由 Spring Boot 调用 backend-ai 完成基础语义召回,再由 Spring Boot 进行业务重排。LLM 相关操作由 Spring Boot 读取 active 配置后调用外部 API,并在本地完成校验和降级。

4.3 接口设计

4.3.1 REST API 设计原则

系统接口采用 REST 风格, 路径以 /api 开头, 返回统一结构:

```
{
  "code": 200,
  "message": "success",
  "data": {}
}
```

需要登录的接口通过 Authorization: Bearer <token> 传递登录态。管理员接口在后端通过 UserContext.isAdmin() 判断角色, 普通用户访问时返回 403。

4.3.2 主要业务接口

表 4-1 Spring Boot 主要接口

模块	方法	路径	权限	说明
认证	POST	/api/auth/register	公开	用户注册
认证	POST	/api/auth/login	公开	用户登录
文献	GET	/api/literatures/search	公开	普通检索和高级检索
文献	GET	/api/literatures/{id}	公开	文献详情
文献	POST/PUT/DELETE	/api/literatures	管理员	文献维护
收藏	GET/POST/DELETE	/api/favorites	登录用户	收藏列表、收藏、取消收藏
分类	GET	/api/categories	公开	分类列表
分类	POST/PUT/DELETE	/api/categories	管理员	分类维护
智能检索	GET	/api/ai/health	公开	backend-ai 健康检查
智能检索	POST	/api/ai/semantic-search	公开	智能语义

				检索
智能检索	GET	/api/ai/recommend/{literatureId}	公开	相似文献推荐
智能索引	POST	/api/ai/build-index	管理员	重建 FAISS 索引
综述	POST	/api/reviews/generate	登录用户	生成综述
综述	GET	/api/reviews/history	登录用户	查看综述记录
管理员	GET	/api/admin/statistics	管理员	真实数据统计
LLM 配置	GET/POST/PUT/DELETE	/api/admin/llm-configs	管理员	LLM 配置管理
LLM 配置	POST	/api/admin/llm-configs/{id}/activate	管理员	设置 active
LLM 配置	POST	/api/admin/llm-configs/{id}/test	管理员	测试连接

图 6-5 LLM API 管理流程图

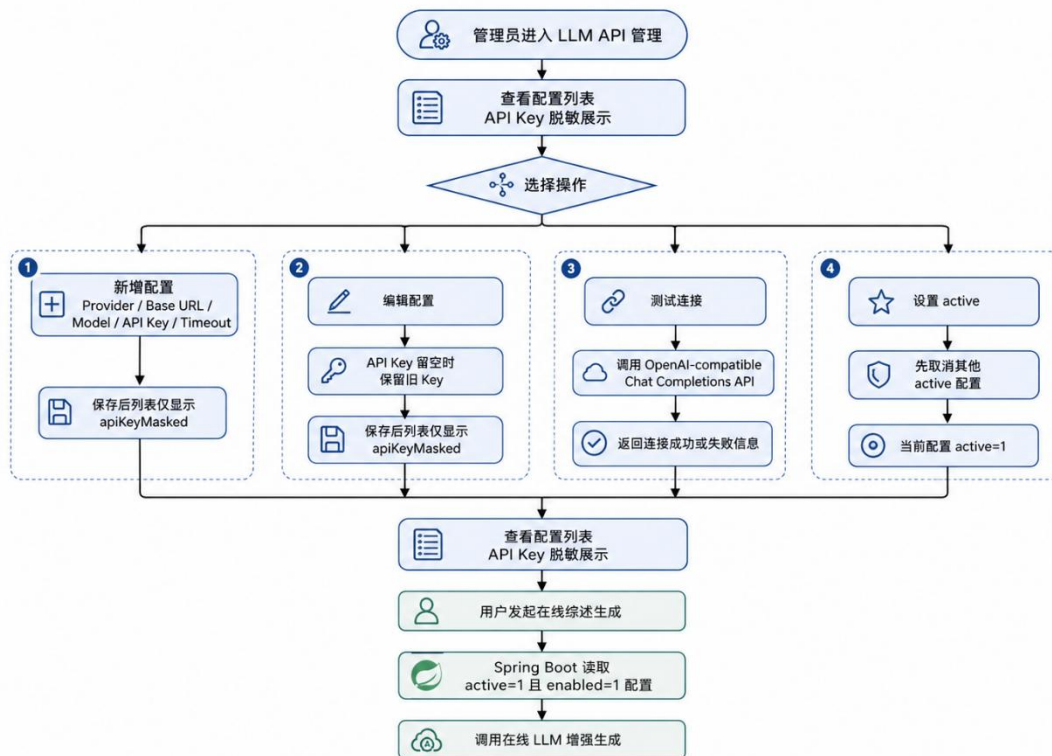


图 6-5 LLM API 管理流程图

LLMAPI 管理流程从新增配置、编辑配置、APIKey 脱敏展示、测试连接、设置 active 配置到在线综述生成读取 active 配置。该流程说明在线增强能力不

是写死在代码中的固定服务，而是由管理员维护的可配置能力。为保证演示安全，前端列表只展示 `apiKeyMasked`，编辑时如果 `API Key` 留空则保留原密钥，避免无意清空配置。

4.3.3 backend-ai 接口

表 4-2 backend-ai 接口

方法	路径	输入	输出	说明
GET	/health	无	服务状态	检查 backend-ai 是否运行
POST	/rebuild-index	documents 文献列表	count	向量化并持久化 FAISS 索引
POST	/semantic-search	query、topK	literatureId、similarity	根据 query 进行语义召回
POST	/recommend	literatureId、topK	literatureId、similarity	根据目标文献向量推荐相似文献

图 6-4 管理员重建智能索引时序图

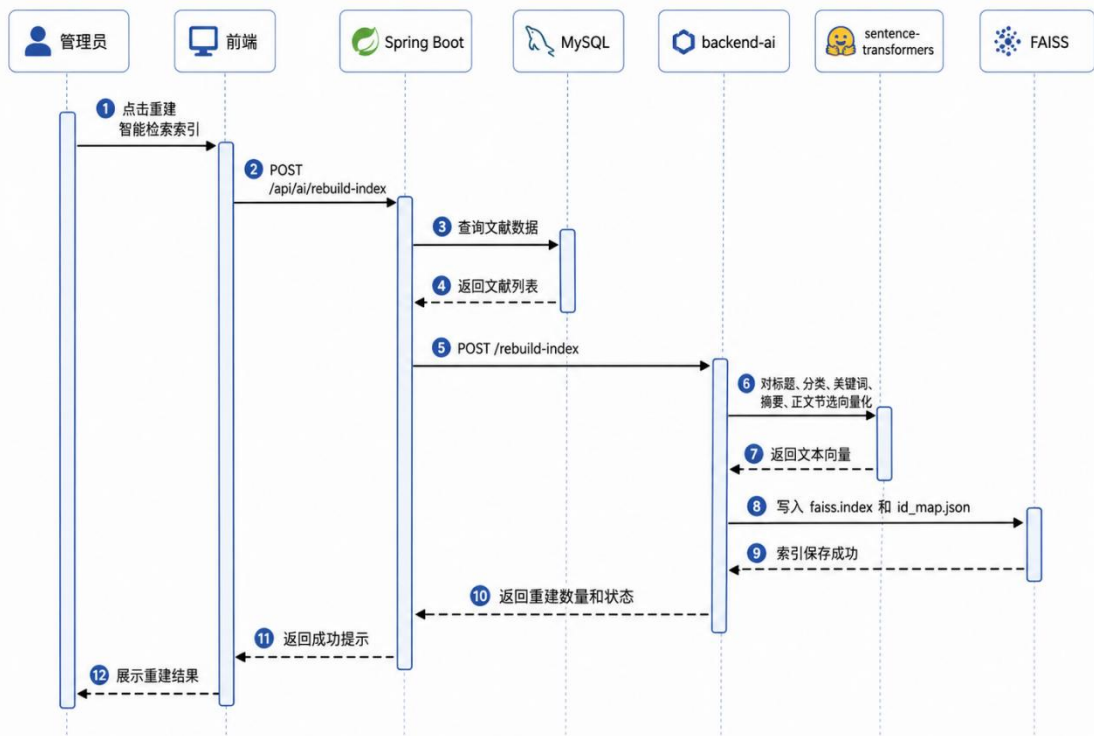


图 6-4 管理员重建智能索引时序图

重建索引时，管理员在前端触发操作，Spring Boot 从 MySQL 查询文献数据并组装 backend-ai 所需的文档列表，backend-ai 使用 sentence-transformers 生成向量，写入 FAISS 索引文件和 ID 映射文件。该时序图体现了 MySQL 与 FAISS 的关系：MySQL 是权威文献数据源，FAISS 是由文献数据派生出的语义

检索索引。

4.4 数据库设计

4.4.1 E-R 图

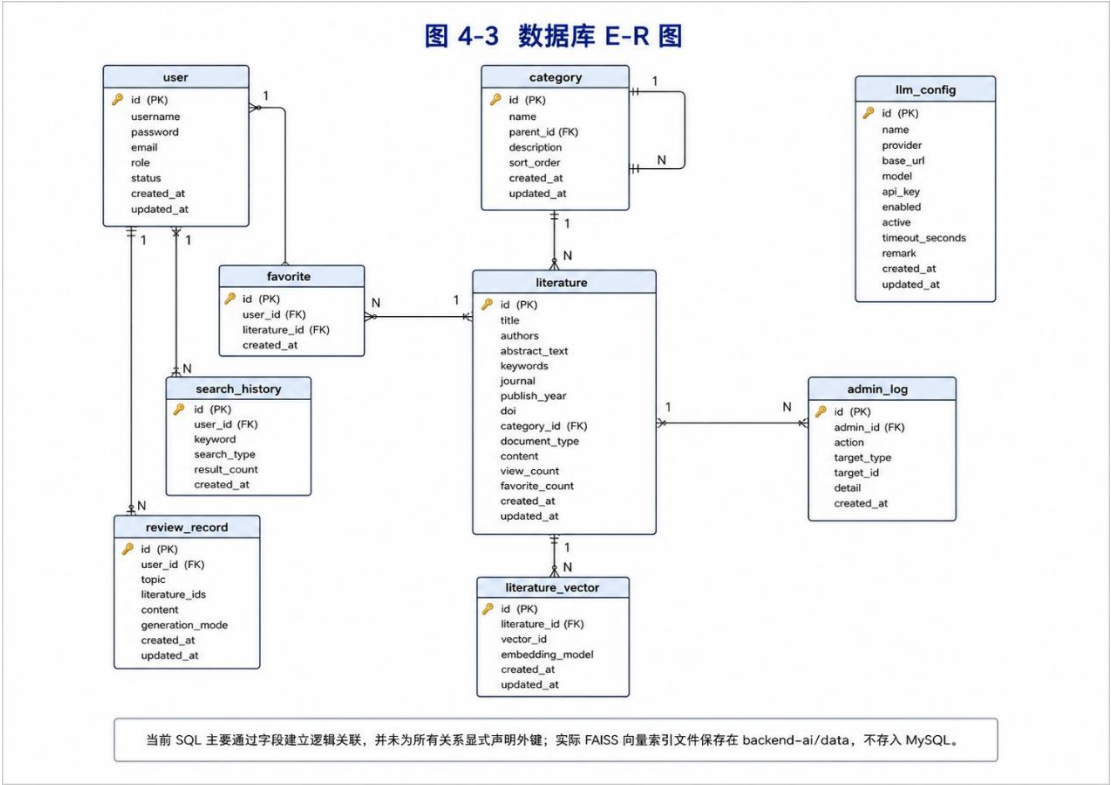


图 4-3 数据库 E-R 图

数据库以 MySQL 为核心，保存用户、分类、文献、收藏、检索历史、综述记录和 LLM 配置。SQL 脚本中未显式声明外键，报告中按逻辑关联说明表关系。FAISS 索引不存入 MySQL，而由 backend-ai 以文件形式保存。

4.4.2 核心表结构

表 4-3 核心数据库表

表名	作用	关键字段
user	用户和管理员账号	id、username、password_hash、role、status
category	两级分类	id、name、parent_id、sort_order
literature	文献元数据	title、authors、abstract_text、keywords、category_id、content
favorite	收藏关系	user_id、

		literature_id, 唯一约束 防止重复收藏
search_history	检索历史	user_id、keyword、 search_type、 result_count
review_record	综述记录	topic、 literature_ids、content、 reference_text、 generation_mode
llm_config	LLM API 配置	provider、 base_url、model、 api_key、active、 timeout_seconds
admin_log	管理员日志	admin_id、 operation、target_type、 target_id
literature_vector	FAISS 映射辅助表	literature_id、 vector_index_id、 embedding_model

4.4.3 两级分类与模拟数据

分类表通过 parent_id 形成两级结构：parent_id=0 表示一级分类，非 0 表示二级分类。项目脚本定义 6 个一级分类和 37 个二级分类，文献表通过 category_id 绑定二级分类。literature_seed_500_high_quality.sql 提供 500 条课程项目模拟文献，用于检索和综述演示。文献数据不是现实论文库，报告和演示中必须明确这一边界。

5 详细设计

5.1 模块算法设计

5.1.1 普通检索与高级检索算法

普通检索使用关键词匹配标题、摘要、关键词和正文节选；高级检索根据用户填写的非空字段动态组合条件。排序支持年份升降序、引用次数升降序和默认排序。该算法简单稳定，适合用户明确知道关键词或筛选条件的场景。

伪代码 5-1 普通检索与高级检索

输入: *LiteratureSearchRequest*

输出: *PageResult<LiteratureListVO>*

page = *request.page* 或默认 1

size = *request.size* 或默认 10

wrapper = 新建查询条件

if *keyword* 非空:

 在 *title*、*abstractText*、*keywords*、*content* 中执行 *like* 匹配

if *title* 非空: *wrapper.like(title)*

if *author* 非空: *wrapper.like(authors)*

if *journal* 非空: *wrapper.like(journal)*

if *doi* 非空: *wrapper.like(doi)*

if *categoryId* 非空: *wrapper.eq(categoryId)*

if *documentType* 非空: *wrapper.eq(documentType)*

if *startYear/endYear* 非空: *wrapper.ge/le(publishYear)*

else if *year* 非空: *wrapper.eq(publishYear)*

按 *sortBy* 设置排序

执行分页查询

批量查询分类名称

转换为 *LiteratureListVO* 返回

5.1.2 智能检索重排算法

图 5-4 智能检索重排算法流程图



图 5-4 智能检索重排算法流程图

智能检索的核心是“语义召回 + 主题增强 + 关键词补充 + 综合重排”。
backend-ai 只负责基础向量召回，Spring Boot 负责业务层面的多学科增强。这样做的原因是课程模拟文献覆盖多学科，单纯语义相似度容易因“人工智能”等宽泛词发生跑偏，因此需要结合学科主题画像和分类权重。

图 6-1 智能检索流程图



图 6-1 智能检索流程图

伪代码 5-2 智能检索重排

输入: *query*, *topK*

输出: 排序后的智能检索结果

```
recallTopK = min(max(topK * 5, 100), 150)
faissResults = 调用 backend-ai /semantic-search(query, recallTopK)
themes = detectThemes(query)
extraLiteratures = mysqlRecallByThemes(themes)
```

候选池 = FAISS 召回文献 + MySQL 补充召回文献

对候选池去重

```
expansionWords = getExpansionWords(themes)
```

```
queryTokens = extractQueryTokens(query, expansionWords)
```

for each literature in 候选池:

semanticSimilarity = FAISS 相似度, 若为补充召回则默认 0.45

keywordScore = 标题/关键词/摘要/正文/分类命中得分

categoryScore = 主题画像对应分类权重

weakPenalty = 弱相关分类降权

finalScore = semanticSimilarity * 0.60

+ keywordScore * 0.30

+ categoryScore * 0.10

- weakPenalty

matchReason = 语义相似度、主题、命中字段、扩展词、分类和降权解释

按 finalScore 降序排序

返回 topK 个结果

5.1.3 backend-ai 向量检索算法

backend-ai 使用 sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2 生成向量。文献向量化文本中标题和关键词会重复一次, 以提高其在语义表示中的权重。向量经 L2 归一化后写入 FAISS IndexFlatIP, 此时内积可近似表示余弦相似度。相似推荐通过目标文献在 FAISS 中的位置还原向量, 检索 topK+1 后排除自身。

5.1.4 综述生成算法

图 6-2 综述生成流程图

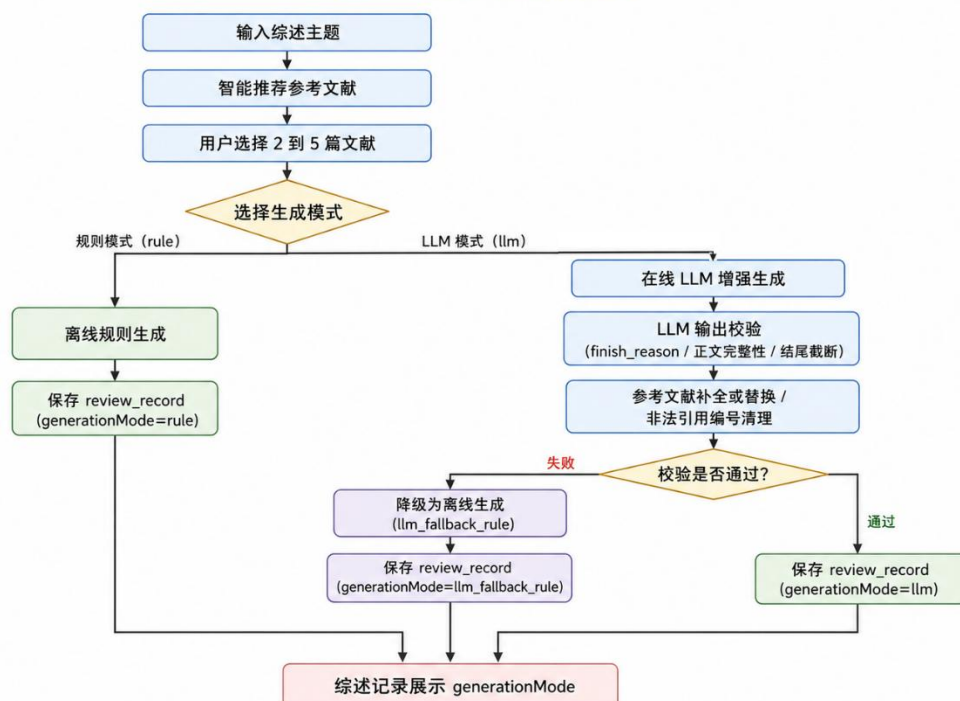


图 6-2 综述生成流程图

综述生成算法先校验主题和文献数量，要求至少 2 篇、最多 5 篇。`rule` 模式执行本地规则生成，包含主题识别、关键词提取、方向归纳、研究背景、研究现状、主要方向、存在问题、发展趋势和参考文献来源；`llm` 模式读取 `active LLM` 配置并调用外部 `API`，生成后进行校验；失败时自动执行本地规则生成并记录 `llm_fallback_rule`。

图 6-6 RAG 风格综述生成流程图

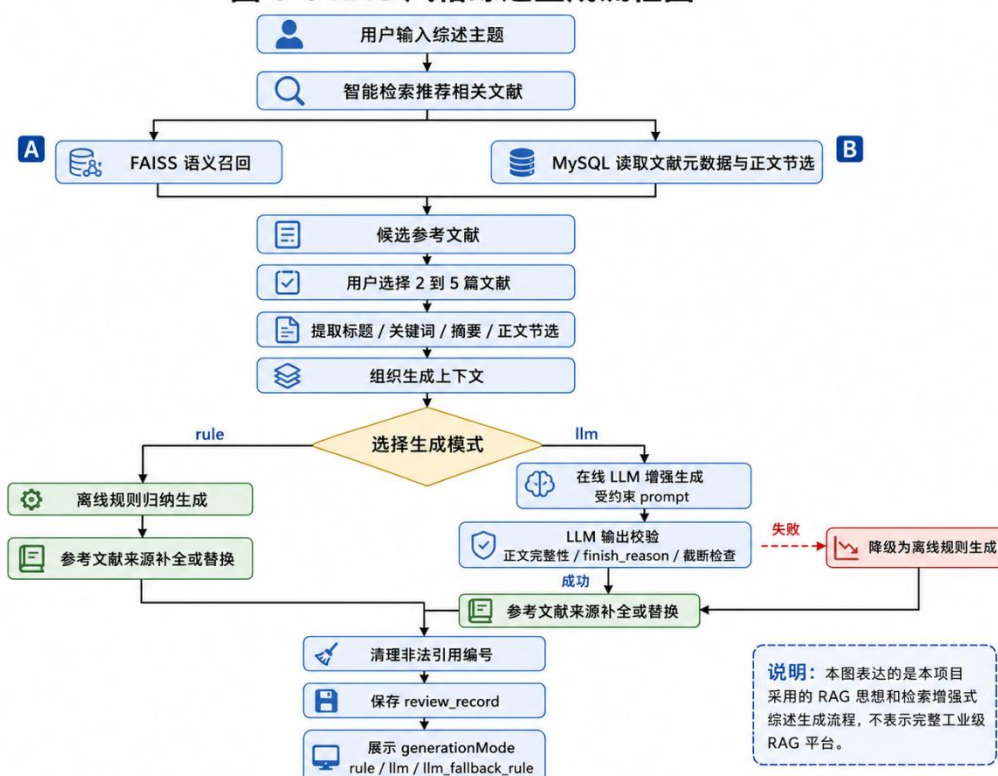


图 6-6 RAG 风格综述生成流程图

伪代码 5-3 综述生成双模式

输入: topic, literatureIds, mode

输出: ReviewRecordVO

if topic 为空: 抛出参数错误

if literatureIds 数量 < 2: 抛出参数错误

if literatureIds 数量 > 5: 抛出参数错误

literatures = 按 ID 查询文献

if 有文献不存在: 抛出参数错误

if mode == "llm":

activeConfig = 查询 active LLM 配置

if activeConfig 不存在或 API Key 为空:

content = buildReviewContent(topic, literatures)

generationMode = "llm_fallback_rule"

else:

try:

content = llmReviewService.generateReview(topic, literatures, activeConfig)

generationMode = "llm"

catch:

content = buildReviewContent(topic, literatures)

generationMode = "llm_fallback_rule"

else:

content = buildReviewContent(topic, literatures)

generationMode = "rule"

保存 `review_record(topic, literatureIds, content, referenceText, generationMode)`
返回综述内容、参考文献和 `generationMode`

5.1.5 LLM 输出校验与降级算法

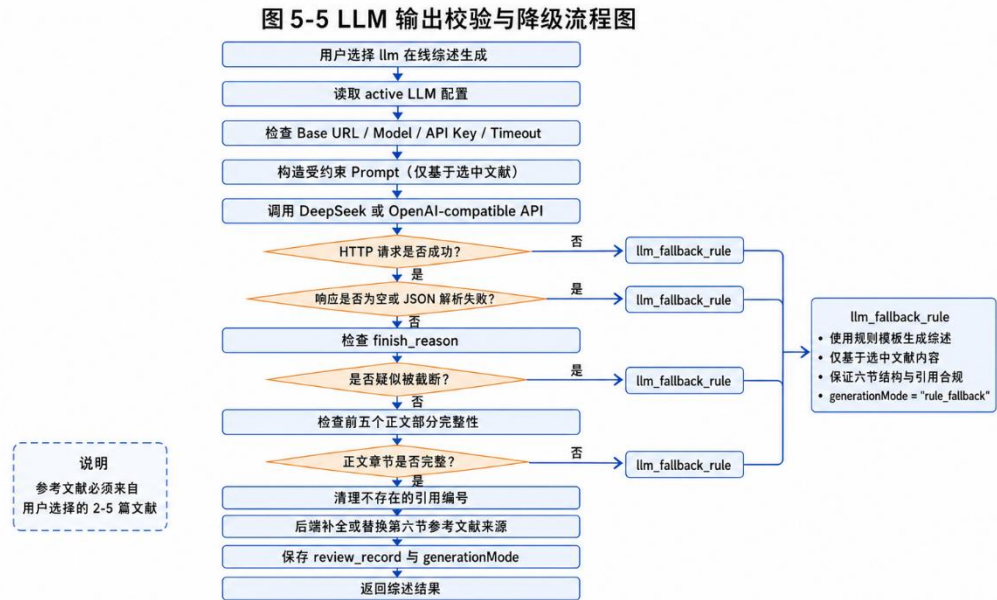


图 5-5 LLM 输出校验与降级流程图

LLM 输出校验是 RAG 风格流程中的约束环节。系统要求 LLM 只能依据输入文献材料生成内容，输出必须包含“研究背景、研究现状、主要研究方向、存在问题、发展趋势、参考文献来源”六部分。后端会检查 `finish_reason`，如果因长度截断会提高 `max_tokens` 重试一次；若仍截断则降级。系统还检查前五个正文部分是否完整，检查结尾是否明显截断，并强制补全或替换第六节参考文献来源。非法引用编号会被清理，参考文献必须来自用户选择的文献。

图 6-3 在线 LLM 综述生成时序图

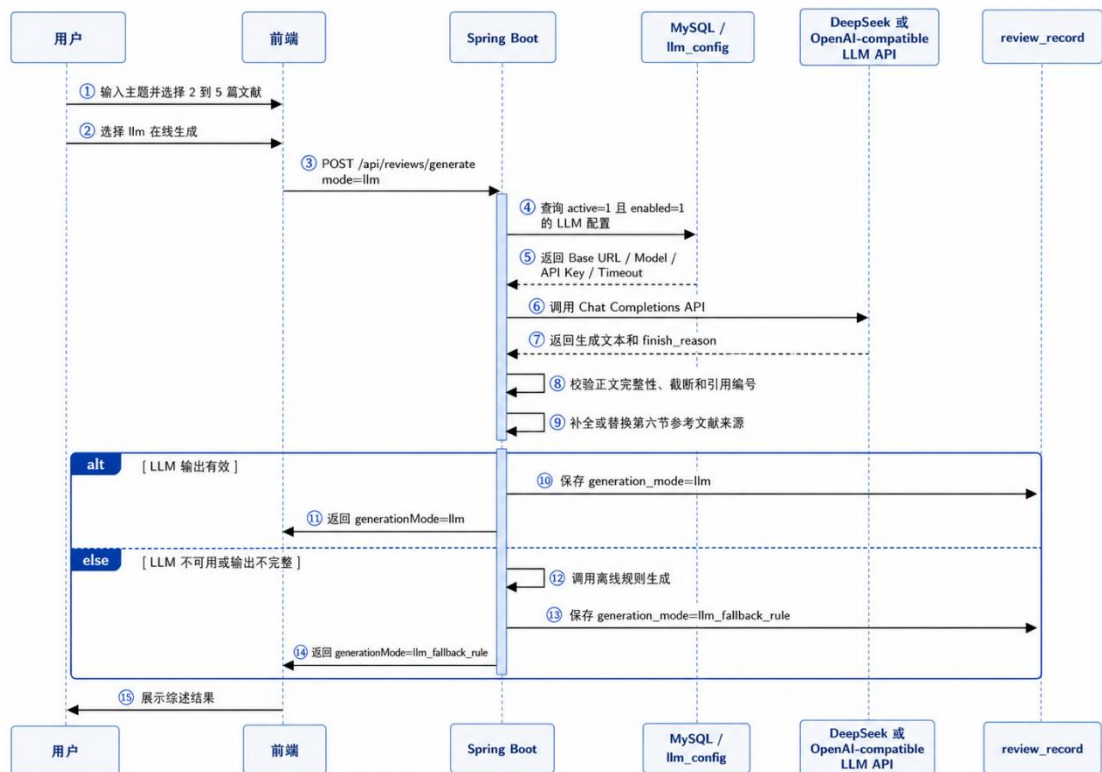


图 6-3 在线 LLM 综述生成时序图

在线 LLM 综述生成时，用户在前端选择在线模式，Spring Boot 读取 MySQL 中 active 的 llm_config，将用户选定文献组织为受约束 prompt，调用 DeepSeek 或 OpenAI-compatible LLM API。LLM 返回后，Spring Boot 先校验输出，再写入 review_record。如果任一环节失败，系统执行本地规则生成并记录 llm_fallback_rule。

5.1.6 面向对象类图

图 5-3 面向对象类图

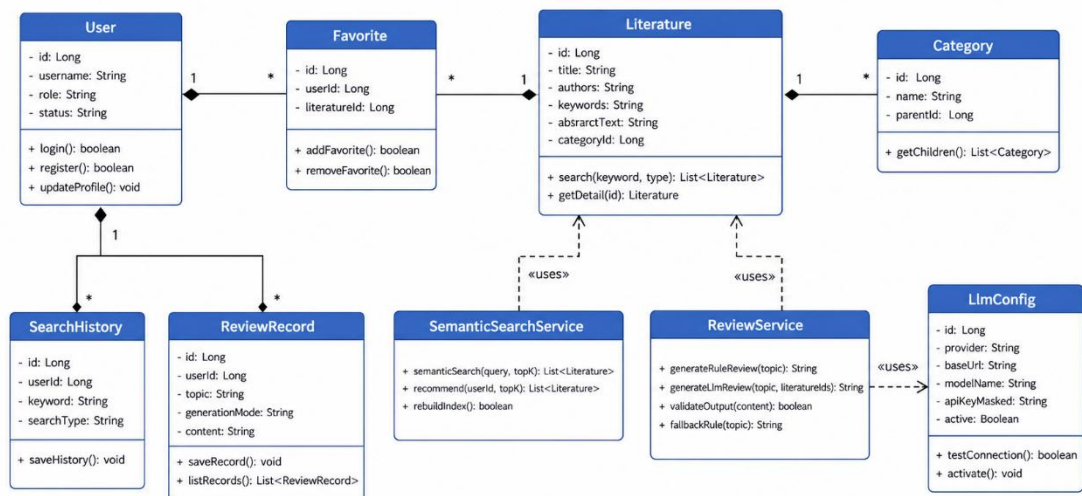


图 5-3 面向对象类图

类图展示了系统中的核心对象和服务关系。User、Category、Literature、Favorite、SearchHistory、ReviewRecord 和 LlmConfig 对应数据库中的主要业务实体；SemanticSearchService 负责智能检索、推荐和索引重建；ReviewService 负责离线综述生成、在线综述生成、输出校验和降级。该图体现了实体类、业务服务和外部服务之间的协作关系，有助于从面向对象角度理解代码结构。

5.2 输入/输出设计：界面原型设计

5.2.1 用户输入设计

表 5-1 主要输入设计

页面	输入项	校验或限制
登录页	用户名、密码	必填，错误时提示
注册页	用户名、密码、姓名、邮箱	用户名和密码必填
普通检索	keyword、排序、分页	keyword 可为空，分页默认 10 条
高级检索	标题、作者、期刊、DOI、分类、年份范围、类型	非空字段参与查询
智能检索	query、topK	query 不能为空
综述生成	topic、literatureIds、mode	文献 2 到 5 篇，mode 为 rule 或 llm
LLM 配置	name、provider、baseUrl、model、apiKey、timeout	API Key 列表脱敏展示

5.2.2 用户输出设计

表 5-2 主要输出设计

页面	输出内容	设计说明
首页	分类入口、系统概览	引导用户进入检索流程
检索结果	标题、作者、摘要、关键词、分类、年份	支持用户快速判断相关性
智能检索结果	similarity、finalScore、matchReason	提供相关性解释
文献详情	文献完整元数据、正文节选、相似推荐	支撑收藏和综述材料判断
综述结果	六部分综述、参考文献来源、generationMode	明确生成方式和来源
管理员统计	用户、文献、收藏、综述、检索、LLM 状态	使用真实数据库聚合

5.2.3 界面原型与运行效果

本项目以实际运行页面截图作为 Web 界面原型和运行效果说明。截图在第 9 章用户手册中按操作流程完整展示，包括首页、登录注册、普通检索、高级检索、智能检索、文献详情、收藏、综述生成、综述记录、管理员后台、文献管理、分类管理、用户管理、统计面板、索引重建、LLM 配置和服务启动终端。

5.3 代码规范

5.3.1 后端代码规范

后端按 Java 分层结构组织，controller 只负责参数接收、权限判断和响应封装；service 负责业务规则；mapper 负责数据库访问；entity、dto、vo 分别对应数据库实体、请求对象和响应对象。异常通过 BusinessException 和统一结果结构返回。命名上使用驼峰命名，数据库字段通过 MyBatis-Plus 下划线转驼峰映射。

5.3.2 前端代码规范

前端按页面、组件和 API 封装组织。接口调用集中在 src/api，页面路由集中在 src/routes。表单输入先在前端给出基本校验，例如综述生成要求至少选择 2 篇参考文献、最多 5 篇。涉及敏感信息的页面只展示后端返回的 apiKeyMasked，不在列表中显示完整 API Key。

5.3.3 文档与安全规范

文档中只说明配置项含义，不写真实 APIKey、数据库密码或 Token。数据库脚本中的初始化账号仅服务课程演示，正式环境应替换为安全密码和环境变量配置。报告中描述 LLM API 时强调在线模式是增强模式，离线规则生成是默认稳定模式。

6 编码实现

6.1 开发环境

表 6-1 开发与运行环境

项目	环境
操作系统	Windows 10 / 11
前端	Node.js、React 19、Vite 7、TypeScript
主后端	Java 17、Spring Boot 3.2、Maven、MyBatis-Plus 3.5.5
数据库	MySQL 8.x
智能检索服务	Python、FastAPI、uvicorn、sentence-transformers、faiss-cpu、numpy
浏览器	Chrome / Edge
LLM API	DeepSeek / OpenAI-compatible API，可选配置

6.2 核心代码片段

6.2.1 普通检索和高级检索代码

```
源          文          件          :          backend-
java/src/main/java/com/campus/literature/service/impl/LiteratureServiceImpl.java
    if (StringUtils.hasText(request.getKeyword())) {
        wrapper.and(w -> w.like(Literature::getTitle, request.getKeyword
        ()))
            .or()
            .like(Literature::getAbstractText, request.getKeyword())
            .or()
            .like(Literature::getKeywords, request.getKeyword())
            .or()
            .like(Literature::getContent, request.getKeyword());
    }
    if (StringUtils.hasText(request.getTitle())) {
        wrapper.like(Literature::getTitle, request.getTitle());
    }
    if (StringUtils.hasText(request.getAuthor())) {
        wrapper.like(Literature::getAuthors, request.getAuthor());
    }
    if (request.getStartYear() != null || request.getEndYear() != null) {
        if (request.getStartYear() != null) {
            wrapper.ge(Literature::getPublishYear, request.getStartYear
            ());
        }
    }
}
```

```

    }
    if (request.getEndYear() != null) {
        wrapper.le(Literature::getPublishYear, request.getEndYear());
    }
}

```

该代码体现了普通关键词检索和高级检索共用同一个查询入口。普通检索将 keyword 同时匹配标题、摘要、关键词和正文节选；高级检索只把用户填写的字段加入查询条件。

6.2.2 智能检索重排代码

```

源文件： backend-
java/src/main/java/com/campus/literature/service/impl/AiServiceImpl.java
    int recallTopK = Math.min(Math.max(topK * 5, 100), 150);
    requestBody.put("query", query);
    requestBody.put("topK", recallTopK);

    List<Map<String, Object>> faissResults = callAiSearchApi(url, request
    Body);
    Set<TopicProfile> detectedThemes = detectThemes(query);
    List<Literature> extraLiteratures = mysqlRecallByThemes(detectedThem
    es);
    List<RankedItem> ranked = scoreAndRank(faissResults, extraLiterature
    s, query, detectedThemes);

    double finalScore = semanticSimilarity * 0.60
        + ks.score * 0.30
        + categoryScore * 0.10;

    double weakPenalty = computeWeakPenalty(categoryName, allRelevantCate
    gories, hitsExpansion, themes);
    finalScore = Math.max(0.0, finalScore - weakPenalty);

    vo.setSimilarity(semanticSimilarity);
    vo.setFinalScore(finalScore);
    vo.setMatchReason(reason.toString());

```

该代码体现了项目智能检索的真实实现：先扩大召回，再补充 MySQL 关键词召回，最后按业务规则计算 finalScore。matchReason 能向用户解释结果来源和降权原因。

6.2.3 backend-ai FAISS 索引代码

```

源文件： backend-ai/main.py
    texts = [build_vector_text(doc) for doc in request.documents]
    embeddings = st_model.encode(texts, show_progress_bar=False, convert_
    to_numpy=True)
    embeddings = normalize_embeddings(embeddings)

    dim = embeddings.shape[1]
    index = faiss.IndexFlatIP(dim)

```

```
index.add(embeddings.astype(np.float32))
```

```
ids = [doc.id for doc in request.documents]
save_index(index, ids)
faiss_index = index
id_map = ids
```

该代码说明 FAISS 索引并不存储在 MySQL 中，而是由 backend-ai 持久化为 faiss.index 和 id_map.json。Spring Boot 负责把数据库文献传给 backend-ai 重建索引。

6.2.4 综述生成双模式代码

源 文 件 : backend-
java/src/main/java/com/campus/literature/service/impl/ReviewRecordServiceImpl.java

```
    if (literatureIds == null || literatureIds.size() < 2) {
        throw new BusinessException(ErrorCode.PARAM_ERROR.getCode(), "请
至少选择 2 篇参考文献");
    }
    if (literatureIds.size() > 5) {
        throw new BusinessException(ErrorCode.PARAM_ERROR.getCode(), "最
多选择 5 篇参考文献");
    }

    boolean useLlm = "llm".equalsIgnoreCase(mode);
    if (useLlm) {
        LlmConfig activeConfig = llmConfigMapper.selectActive();
        if (activeConfig == null || activeConfig.getApiKey() == null || a
ctiveConfig.getApiKey().isEmpty()) {
            content = buildReviewContent(topic, literatures);
            generationMode = "llm_fallback_rule";
        } else {
            try {
                content = llmReviewService.generateReview(topic, literatur
es, activeConfig);
                generationMode = "llm";
            } catch (Exception e) {
                content = buildReviewContent(topic, literatures);
                generationMode = "llm_fallback_rule";
            }
        }
    } else {
        content = buildReviewContent(topic, literatures);
        generationMode = "rule";
    }
}
```

该代码体现了综述生成的边界条件、在线模式和降级机制。即使 LLM 失败，系统仍能生成离线综述并记录降级模式。

6.2.5 LLM 输出校验代码

```
源          文          件          :          backend-
java/src/main/java/com/campus/literature/service/impl/LlmReviewServiceImpl.java
    LlmResult first = callLlm(url, config, model, userPrompt, timeout
Seconds, 2500);
    String content = first.content;
    boolean needsRetry = "length".equals(first.finishReason);

    if (needsRetry) {
        LlmResult second = callLlm(url, config, model, userPrompt, timeou
tSeconds, 3000);
        content = second.content;
        if ("length".equals(second.finishReason)) {
            throw new RuntimeException("FINISH_REASON_LENGTH: 两次均因 max
_tokens 截断");
        }
    }

    String[] bodySections = {"研究背景", "研究现状", "主要研究方向", "存在问
题", "发展趋势"};
    int bodySectionCount = 0;
    for (String s : bodySections) {
        if (content.contains(s)) bodySectionCount++;
    }
    if (bodySectionCount < 5) {
        throw new RuntimeException("MISSING_SECTION");
    }

    content = ensureReferencesComplete(content, literatures);
    content = cleanInvalidCitations(content, literatures.size());
```

该代码体现了系统不是简单调用 LLM 后直接展示，而是检查截断、章节完整性、参考文献来源和引用编号。

6.2.6 API Key 脱敏代码

```
源          文          件          :          backend-
java/src/main/java/com/campus/literature/service/impl/LlmConfigServiceImpl.java
    private LlmConfigVO toVO(LlmConfig config) {
        LlmConfigVO vo = new LlmConfigVO();
        vo.setId(config.getId());
        vo.setName(config.getName());
        vo.setProvider(config.getProvider());
        vo.setBaseUrl(config.getBaseUrl());
        vo.setModel(config.getModel());
        vo.setApiKeyMasked(maskApiKey(config.getApiKey()));
        vo.setEnabled(config.getEnabled());
        vo.setActive(config.getActive());
        vo.setTimeoutSeconds(config.getTimeoutSeconds());
        return vo;
    }
```

```
private String maskApiKey(String apiKey) {
    if (apiKey == null || apiKey.length() <= 8) {
        return apiKey != null && !apiKey.isEmpty() ? "*****" : "";
    }
    return apiKey.substring(0, 4) + "*****" + apiKey.substring(apiKey.
length() - 4);
}
```

该代码体现了前端列表只展示脱敏 API Key。生产环境仍应进一步加密存储密钥。

6.2.7 JWT 权限拦截代码

```
源          文          件          :          backend-
java/src/main/java/com/campus/literature/security/JwtInterceptor.java
    if (authHeader != null && authHeader.startsWith("Bearer ")) {
        String token = authHeader.substring(7);
        Claims claims = jwtUtil.parseToken(token);
        if (claims != null) {
            Long userId = Long.valueOf(claims.getSubject());
            String username = claims.get("username", String.class);
            String role = claims.get("role", String.class);
            UserContext.setCurrentUser(new UserContext.UserInfo(userId, u
ername, role));
        } else if (!isPublic) {
            writeError(response, 401, "未登录或登录失效");
            return false;
        }
    } else if (!isPublic) {
        writeError(response, 401, "未登录或登录失效");
        return false;
    }
}
```

该代码说明系统通过 Token 解析用户身份和角色，并把当前用户信息写入线程上下文，后续 controller 和 service 可通过 UserContext 判断权限。

6.3 代码结构说明

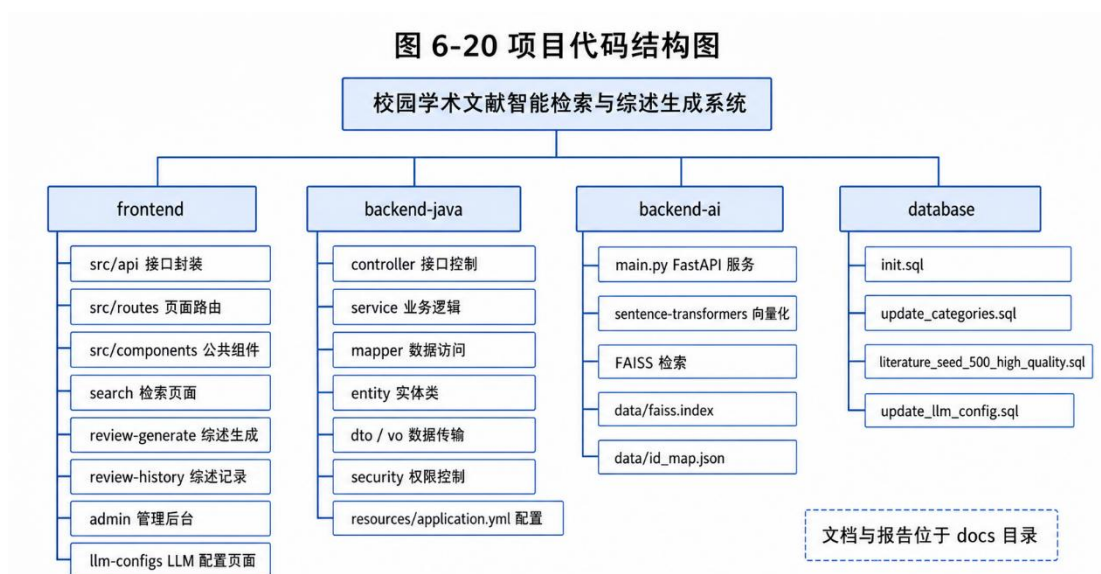


图 6-20 项目代码结构图

项目根目录按前端、后端、智能检索服务、数据库和文档分离组织。

表 6-2 项目代码结构说明

目录	说明
frontend	React + Vite 前端项目，包含 src/api、src/routes、src/components
backend-java	Spring Boot 主后端，包含 controller、service、mapper、entity、dto、vo、security
backend-ai	FastAPI 智能检索服务，核心文件为 main.py
database	MySQL 建表、分类更新、模拟文献导入和 LLM 配置更新脚本
docs	项目报告、接口文档、数据库设计、测试用例、图表和截图

7 测试

7.1 测试计划

测试目标是验证系统能够完成从启动、登录、检索、推荐、综述生成、后台管理到异常降级的完整流程。测试分为构建测试、功能测试、接口测试、边界测试、异常测试和回归测试。测试环境包括 MySQL、backend-ai、backend-java 和 frontend 四部分，必要时配置 DeepSeek / OpenAI-compatible LLM API 进行在线生成测试。

图 7-1 测试与缺陷闭环图

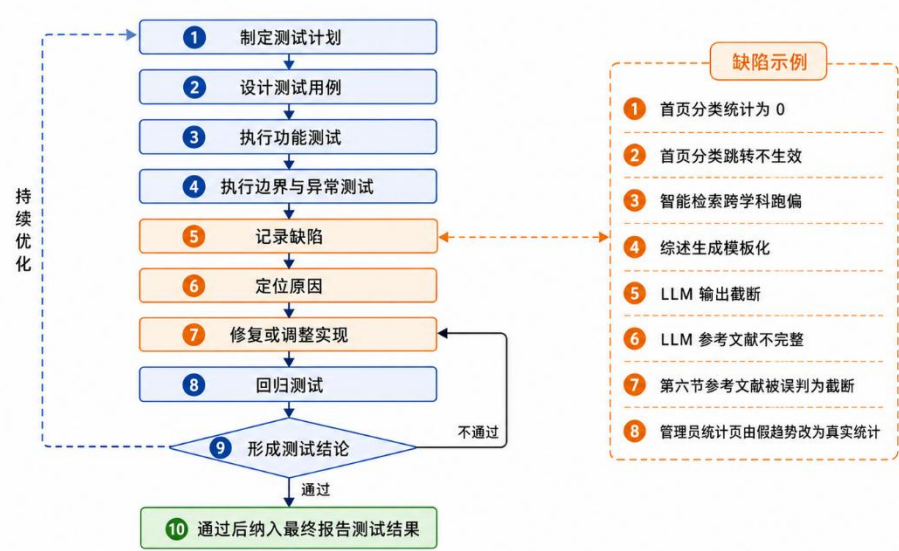


图 7-1 测试与缺陷闭环图

测试闭环包括制定测试计划、设计测试用例、执行功能测试、执行边界与异常测试、记录缺陷、定位原因、修复或调整实现、回归测试和形成测试结论。该流程保证缺陷不只被记录，还能回到实现和测试中验证。

7.2 测试用例

表 7-1 核心测试用例

编号	测试项	操作步骤	预期结果
TC-BUILD-001	backend-ai 语法检查	执行 <code>python -m py_compile main.py</code>	无语法错误
TC-BUILD-002	backend-java 编译	执行 <code>mvn clean package -DskipTests</code>	构建成功
TC-BUILD-	frontend 构建	执行 <code>npm run build</code>	构建成功

003			
TC-AUTH-001	用户登录	输入正确用户名和密码登录	登录成功并保存 Token
TC-AUTH-002	密码错误	输入错误密码登录	返回错误提示
TC-LIT-001	普通检索	搜索“人工智能”	返回相关文献
TC-LIT-002	高级检索	按年份、分类、作者组合查询	返回符合条件的文献
TC-AI-001	重建智能索引	管理员点击重建索引	生成 faiss.index 和 id_map.json
TC-AI-002	智能检索	查询“人工智能在医学影像诊断中的应用”	优先返回医学相关文献
TC-AI-003	重排字段	调用智能检索接口	返回 similarity、finalScore、matchReason
TC-REC-001	相似推荐	打开文献详情页	展示相似文献且不包含自身
TC-REV-001	少于 2 篇生成	只选择 1 篇文献	提示至少选择 2 篇
TC-REV-002	超过 5 篇生成	选择第 6 篇文献	阻止选择或提示最多 5 篇
TC-REV-003	离线生成	选择 2 到 5 篇，mode=rule	生成六部分综述，generationMode=rule
TC-REV-004	在线生成	配置 active LLM 后 mode=llm	成功时 generationMode=llm
TC-REV-005	LLM 降级	停用 active 配置后在线生成	自动降级，generationMode=llm_fallback_rule
TC-LLM-001	API Key 脱敏	查看 LLM 配置列表	只显示 apiKeyMasked
TC-ADM-001	普通用户访问后台	普通用户访问管理员接口	返回 403
TC-STAT-001	统计面板	管理员查看统计页	展示数据库真实统计

7.3 缺陷分析

表 7-2 缺陷记录与修复说明

编号	问题	原因	修复方式
BUG-001	首页分类统计为 0	早期统计逻辑未按二级分类真实聚合	后端按全部文献和分类重新聚合
BUG-002	首页分类跳转不生	前端传分类名称，后端需要 categoryId	首页跳转改为传 categoryId，搜索页监听 URL

	效		参数
BUG-003	智能检索跨学科跑偏	纯 FAISS 容易受宽泛词影响	增加主题画像、Query Expansion、MySQL 补充召回和 weakPenalty
BUG-004	综述生成模板化	早期结果接近摘要拼接	增加主题识别、关键词提取、方向归纳、问题和趋势生成
BUG-005	LLM 输出截断	finish_reason=length 或 token 不足	检查 finish_reason，提高 max_tokens 重试，失败降级
BUG-006	LLM 参考文献不完整	模型可能漏写第六节	后端 ensureReferencesComplete 强制补全或替换
BUG-007	第六节参考文献被误判为截断	缺少第六节时过早判定失败	前五节完整即可进入参考文献补全
BUG-008	LLM 产生不存在的引用编号	模型可能输出超出范围编号	后端 cleanInvalidCitations 删除非法引用
BUG-009	管理员统计页存在假趋势数据	早期前端写死展示数据	改为数据库真实统计面板

8 维护计划

8.1 纠错性维护

纠错性维护主要处理运行过程中发现的缺陷,包括接口异常、页面显示错误、检索结果异常、LLM 降级失败、统计数据不一致等。维护流程为:复现问题、记录输入条件和接口响应、定位前端或后端原因、修改实现、执行回归测试、更新测试用例和开发日志。对于智能检索问题,应重点检查 `query`、主题画像、扩展词、分类权重、FAISS 索引是否过期以及 MySQL 补充召回是否生效。

8.2 适应性维护

适应性维护主要面向运行环境变化。MySQL 版本、Java 版本、Node.js 版本、Python 依赖、sentence-transformers 模型下载路径、FAISS 包版本和 LLM API 兼容格式都可能变化。维护时应优先保持接口字段和数据库表结构稳定,必要时新增脚本或迁移说明。若更换 LLM 服务,只需在管理员端配置新的 Base URL、Model 和 API Key,避免修改业务代码。

8.3 完善性维护

完善性维护包括提升检索质量、优化综述生成质量、完善统计维度和增强用户体验。后续可考虑引入 chunk 切分、向量数据库、段落级引用溯源、Cross-Encoder rerank、BM25+ 向量混合检索、LLM 流式输出、PDF 上传与全文解析、生产级 API Key 加密存储等能力。这些功能当前未实现,只能作为后续展望。

9 用户手册

9.1 安装说明

9.1.1 数据库准备

先启动 MySQL，创建数据库并执行脚本。建议顺序为：database/init.sql、database/update_categories.sql、database/literature_seed_500_high_quality.sql、database/update_llm_config.sql。如果只执行 init.sql，需要注意 review_record.generation_mode 和 llm_config 可能尚未更新。

9.1.2 backend-ai 启动

```
cd backend-ai
pip install -r requirements.txt
uvicorn main:app --reload --port 8000
```

启动后访问 <http://localhost:8000/health>，返回 status=ok 表示服务运行正常。首次运行 sentence-transformers 可能需要下载模型，耗时取决于网络环境。

9.1.3 backend-java 启动

```
cd backend-java
mvn spring-boot:run
```

后端默认端口为 8080。数据库连接、JWT 密钥和其他敏感配置应在本地配置文件或环境变量中设置，正式报告不展示真实密码或 Token。

9.1.4 frontend 启动

```
cd frontend
npm install
npm run dev
```

前端默认端口为 5173，浏览器访问 <http://localhost:5173> 即可进入系统。

9.2 使用指南

9.2.1 普通用户使用流程



图 9-1 首页

首页展示系统入口和分类信息，用户可以从首页进入检索流程。首页分类跳转已经修复为按 `categoryId` 传参，能够直接进入对应分类检索结果。

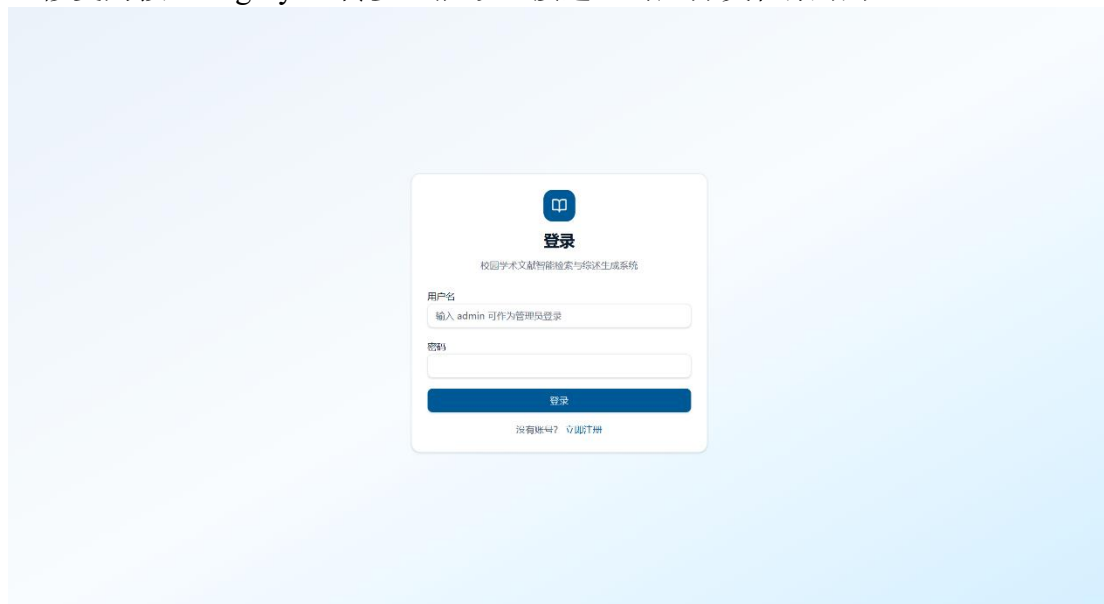


图 9-2 登录页面

用户在登录页输入账号和密码。登录成功后，前端保存 `Token` 并在需要登录的接口中携带请求头。

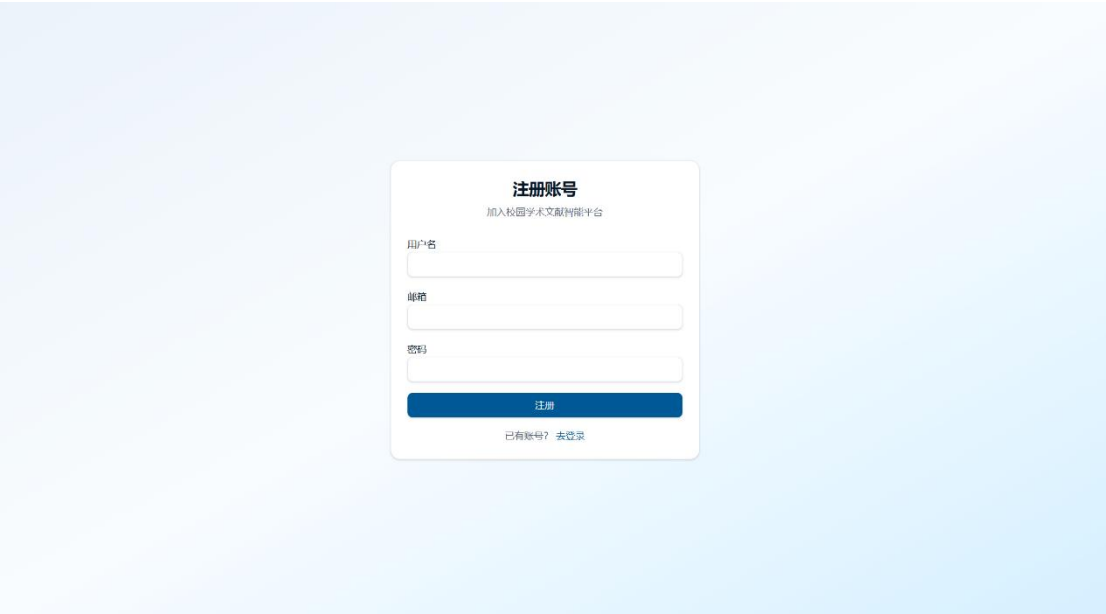


图 9-3 注册页面

新用户可以在注册页面创建普通用户账号。注册成功后可返回登录页进入系统。



图 9-4 普通检索页面

普通检索适合输入明确关键词。结果列表展示标题、作者、摘要、年份、分类等信息，用户可点击进入详情页。



图 9-5 高级检索页面

高级检索适合按标题、作者、期刊、DOI、年份、分类和文献类型进行组合筛选。



图 9-6 智能检索结果页面

智能检索适合输入自然语言主题。结果中展示综合重排信息，帮助用户理解为什么某篇文献被推荐。



图 9-7 文献详情与相似推荐页面

文献详情页展示完整元数据和正文节选，并展示相似文献推荐。相似推荐由 backend-ai 根据目标文献向量检索得到，并排除当前文献本身。



图 9-8 我的收藏页面

用户可以在详情页收藏文献，并在我的收藏页面统一查看。收藏表通过唯一约束防止同一用户重复收藏同一文献。



图 9-9 综述生成双模式选择页面

综述生成页面要求用户输入主题并选择 2 到 5 篇参考文献。用户可以选择离线 rule 模式或在线 llm 模式。

智能综述生成

综述主题

人工智能在法学中的应用

智能推荐参考文献

推荐文献列表

(语义相似度由高到低)

融合人工智能的知识产权方法与实践

孙涛、曹睿 · 2021 · 情报科学 · 学位论文 · 法学

法学

知识产权

人工智能

评价指标

数字化转型

语义相似度: 68.6%

面向校园治理的知识产权模型构建

高峰、胡伟、杨帆 · 2024 · 高等教育研究 · 期刊论文 · 法学

法学

知识产权

知识服务

数字化转型

行为分析

语义相似度: 45.8%

基于实证分析的个人信息保护大学生群体研究

陈思、熊伟、许然 · 2021 · 高等教育研究 · 期刊论文 · 法学

法学

个人信息保护

数字化转型

行为分析

语义相似度: 44.0%

基于数据治理视角的聚类分析机制研究

马欣、王博、孙涛 · 2025 · 软件工程与应用 · 会议论文 · 数据科学与数据挖掘

数据科学与数据挖掘

聚类分析

数据治理

系统设计

语义相似度: 45.0%

生成结果

《人工智能在法学中的应用领域研究综述》

生成时间: 2026-05-22 · 生成方式: 离线综述生成

《人工智能在法学中的应用研究综述》

一、研究背景

人工智能在法学中的应用是法学与人工智能交叉领域的新兴议题。随着数字技术的深入应用,数字化转型、知识产权、法学等问题日益突出,亟需从法律与治理角度进行系统性研究。该领域对于规范技术发展、维护公共利益具有重要的制度意义。

二、研究现状

从所选文献看,人工智能在法学中的应用领域的现有研究主要围绕知识产权与数据合规、算法治理与平台监管、人工智能伦理与责任界定等方向展开。在知识产权与数据合规方面,文献[1][2]进行了相关探索,涉及数字化转型、知识产权、法学。整体而言,该领域已从早期理论探索逐步转向实践应用与系统优化,多学科交叉融合的趋势日益明显。

三、主要研究方向

1. 知识产权与数据合规

该方向聚焦于人工智能时代的知识产权保护与数据合规治理。现有研究关注算法创新成果的法律保护、用户数据使用边界以及合规审查机制构建。相关研究如文献[1][2],分别从数字化转型、知识产权、法学等角度展开,为数字时代的知识产权制度完善提供了理论支撑。

图 9-10 离线综述生成结果页面

离线生成不依赖外部 API, 适合稳定演示。结果包含研究背景、研究现状、主要研究方向、存在问题、发展趋势和参考文献来源。

50

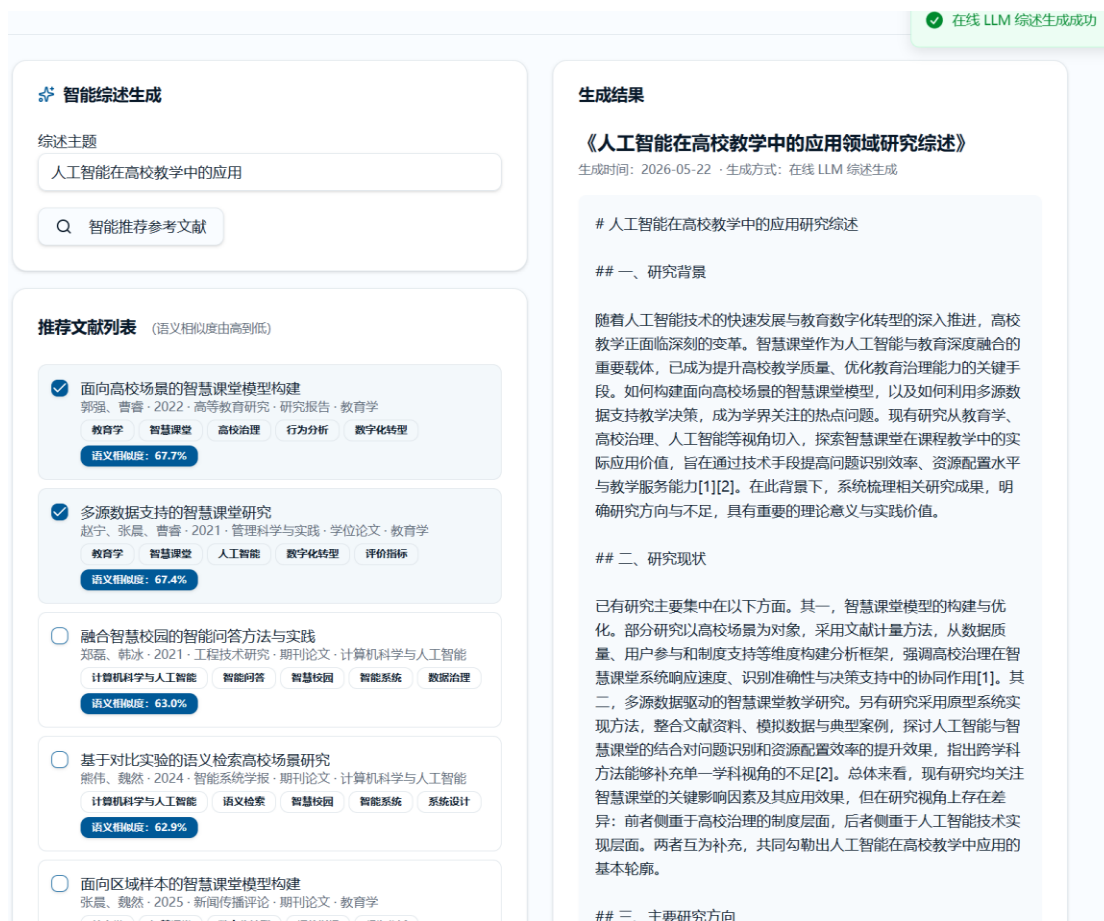


图 9-11 在线 LLM 综述生成结果页面

在线 LLM 模式在 active 配置可用时调用大模型增强表达。结果仍受选中文献材料约束,并由后端补全参考文献来源。



图 9-12 综述记录 generationMode 标签页面

综述记录页展示生成历史,并通过标签区分离线、在线和降级生成方式。



图 9-13 综述记录筛选页面

用户可以按 `generationMode` 筛选历史记录，便于区分不同生成来源。

9.2.2 管理员使用流程



图 9-14 管理员后台首页

管理员后台提供系统维护入口。普通用户不应访问管理员接口。

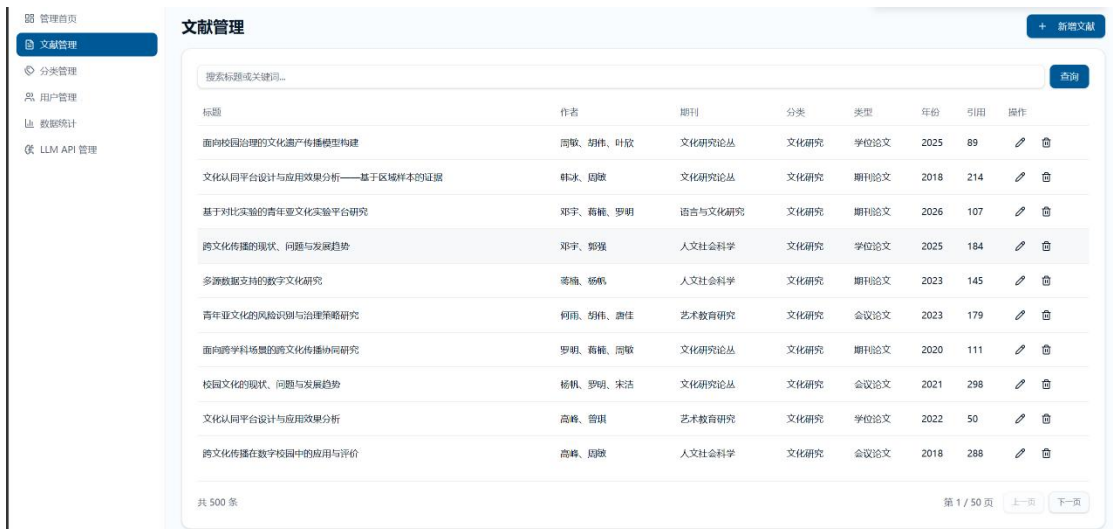


图 9-15 文献管理页面

管理员可新增、编辑、删除文献。文献更新后需要重建智能检索索引，保证 FAISS 索引与 MySQL 数据一致。



图 9-16 分类管理页面

分类管理维护两级学科分类。分类会影响首页展示、高级检索、统计面板和智能检索中的 categoryScore。

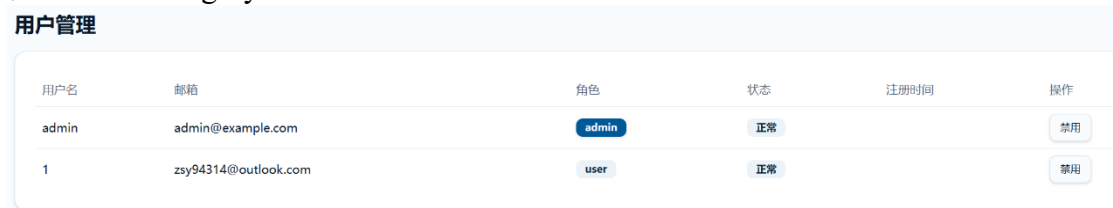


图 9-17 用户管理页面

管理员可查看用户列表并调整用户状态。禁用用户不能继续登录系统。

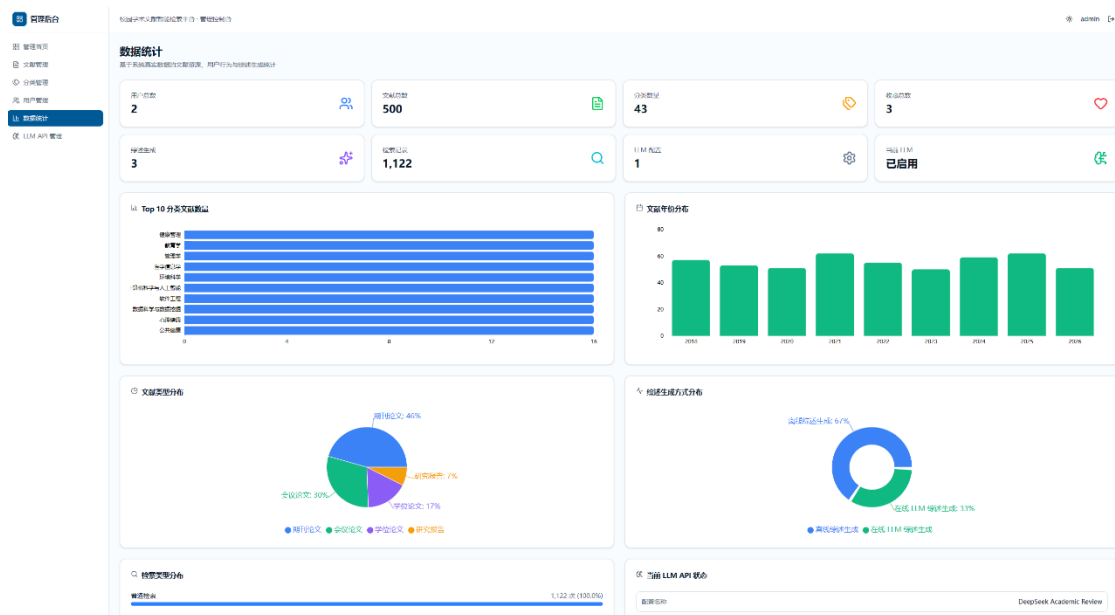


图 9-18 数据统计页面

数据统计页面展示数据库真实聚合结果，包括用户、文献、收藏、综述、检索记录、LLM 配置和分布图表。



图 9-19 重建智能索引入口

文献数据发生变化后，管理员点击重建索引。该操作会读取 MySQL 文献并调用 backend-ai 重建 faiss.index 和 id_map.json。

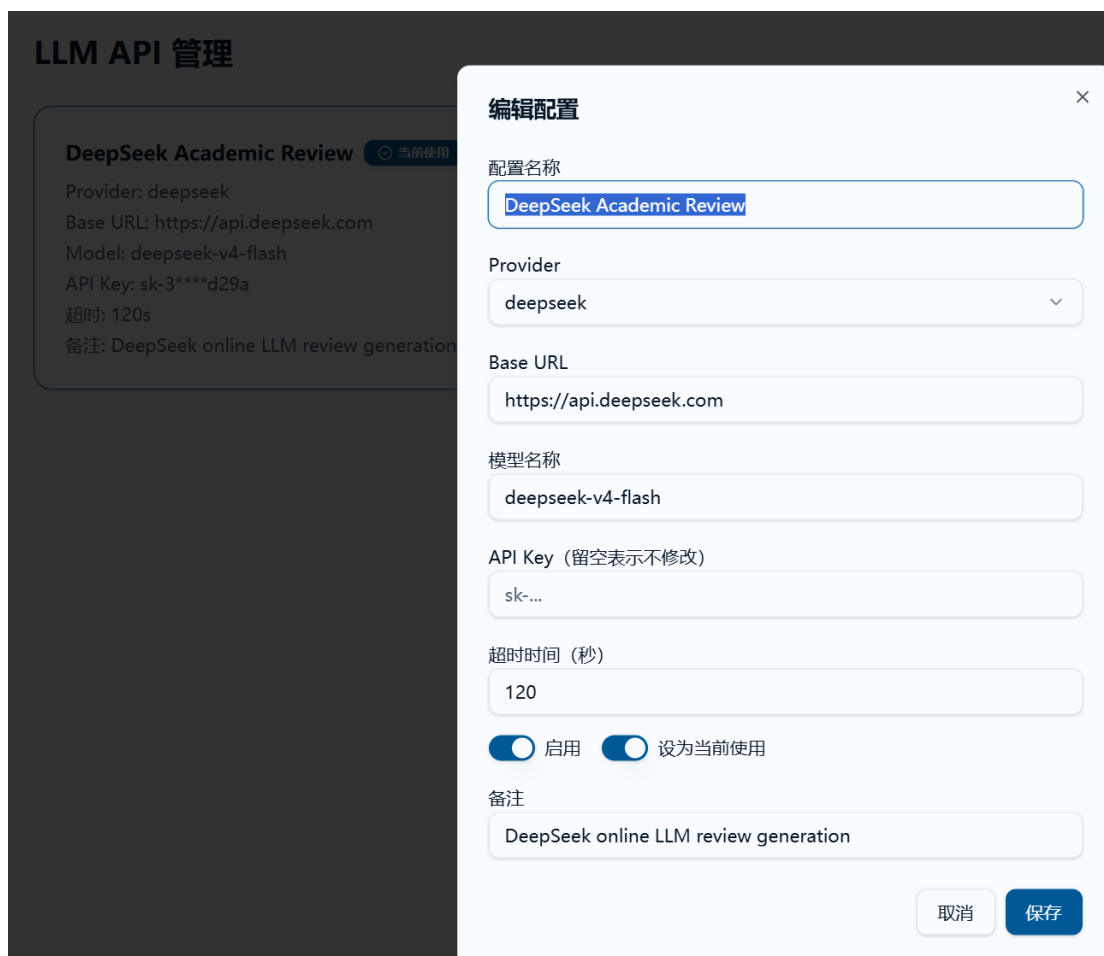
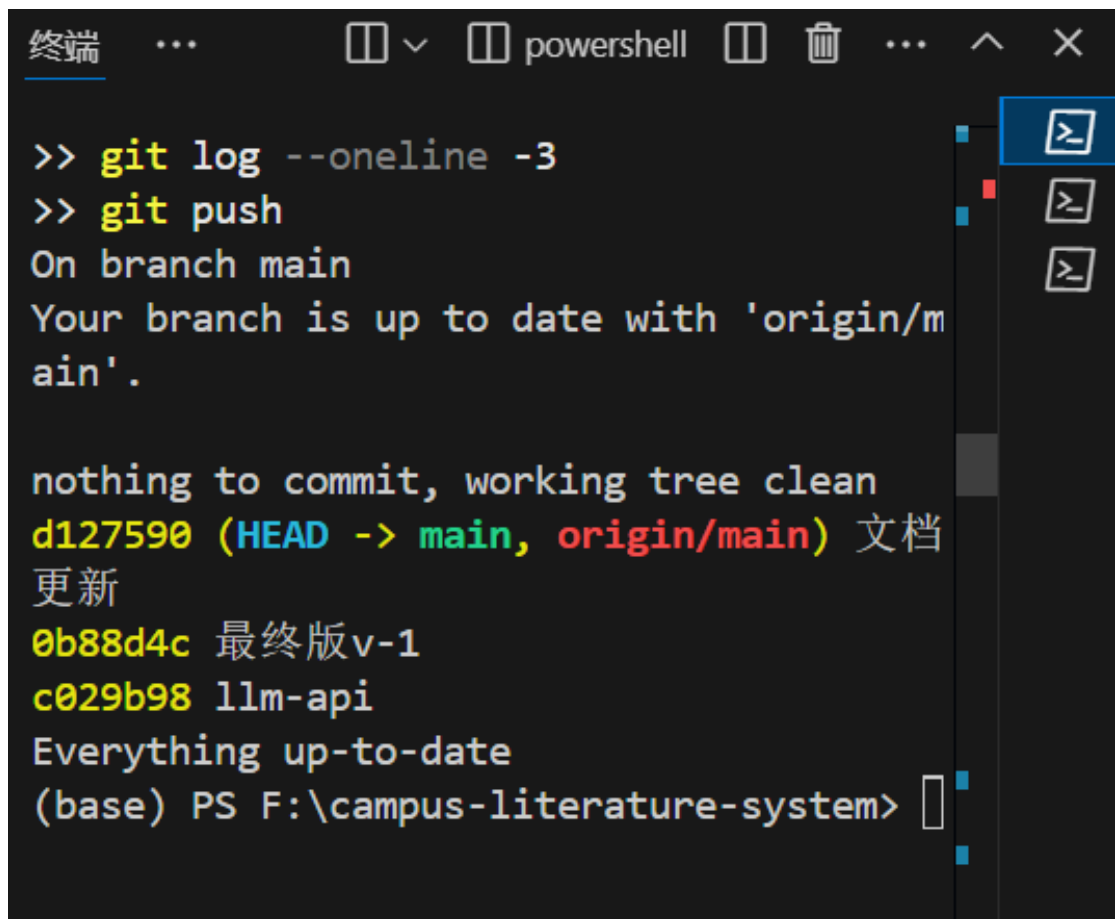


图 9-20 LLM API 管理页面
LLM API 管理页面用于维护 DeepSeek / OpenAI-compatible API 配置。截图中必须遮挡或脱敏 API Key。



图 9-21 LLM 测试连接成功页面
管理员可测试 LLM API 连接。测试成功仅说明当前配置可访问，不代表在线生成一定不会因超时或输出不完整而降级。

A screenshot of a Windows terminal window titled '终端' (Terminal). The window shows the output of two git commands. The first command is 'git log --oneline -3', which displays the last three commits: 'd127590 (HEAD -> main, origin/main) 文档更新', '0b88d4c 最终版v-1', and 'c029b98 llm-api'. The second command is 'git push', which outputs 'On branch main', 'Your branch is up to date with 'origin/main'', 'nothing to commit, working tree clean', and 'Everything up-to-date'. The terminal prompt is '(base) PS F:\campus-literature-system>'. The window has a standard Windows title bar with icons for maximize, minimize, and close, and a search icon. On the right side, there are three icons for terminal operations: a blue icon with a white cursor, a red icon with a white cursor, and a green icon with a white cursor.

```
终端 ... [icon] powershell [icon] [icon] ... ^ X

>> git log --oneline -3
>> git push
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
d127590 (HEAD -> main, origin/main) 文档更新
0b88d4c 最终版v-1
c029b98 llm-api
Everything up-to-date
(base) PS F:\campus-literature-system> [icon]
```

图 9-22 三个服务启动终端

系统运行需要 MySQL、backend-ai、backend-java 和 frontend 协同工作。终端截图用于说明服务启动状态。

10 项目总结

10.1 成果总结

1. 完成了一个前后端分离的校园学术文献智能检索与综述生成系统，覆盖普通用户端和管理员端主要业务流程。
2. 完成了普通检索、高级检索、多学科智能语义检索、相似文献推荐和智能推荐参考文献功能，并通过 `finalScore` 和 `matchReason` 提升检索解释性。
3. 完成了离线 `rule` 和在线 `llm` 双模式综述生成，支持 `llm_fallback_rule` 自动降级、LLM 输出完整性校验、参考文献后端补全和 `generationMode` 记录筛选。
4. 完成了 MySQL 数据库设计、两级分类、500 条课程项目模拟文献、FAISS 索引文件、后台真实统计、索引重建和 LLM API 管理。
5. 完成了软件工程三级项目要求的 DFD、数据字典、SC 图、E-R 图、接口设计、详细设计、核心代码、测试用例、缺陷分析、维护计划和用户手册。

10.2 经验与不足

项目经验主要体现在三个方面。第一，智能功能必须与工程约束结合，不能只追求模型能力。智能检索如果只使用 FAISS 容易跨学科跑偏，综述生成如果只调用 LLM 容易出现来源不可控，因此需要主题画像、重排、降级和校验。第二，课程项目文档必须与真实代码和 SQL 对齐。文档中不能把未实现的 PDF 上传、真实论文库、Docker、流式输出、Cross-Encoder rerank 或生产级密钥加密写成已完成。第三，测试和截图是证明系统真实完成的重要材料，必须把运行界面、核心代码和测试缺陷闭环放入报告正文。

不足之处也比较明确。系统文献数据为模拟数据，不具备真实论文平台的资源规模和版权体系；FAISS 使用本地文件索引，不是专门向量数据库；RAG 流程没有实现复杂 `chunk` 管理和段落级引用溯源；LLM API Key 在课程演示环境中保存配置，生产环境还需要加密存储和审计机制；智能检索重排规则仍然是经验权重，后续可通过人工标注数据进一步优化。

10.3 小组成员分工及自评

表 10-1 小组分工与贡献比例

姓名	学号	角色	主要分工	自评比例
周思源	202411040239	组长	项目总体规划、需求分析、系统架构设计、前后端主体搭建、智能检索与综述生成核心功能实现、前后端联调、项目文档主体撰写、最终汇报与演讲准备	40%
黄文政	202411040232	组员	后端辅助开发，参与 Spring Boot 接口调试、权限控制检查、智能检索接口联调和后端功能测试	15%
李俊宇	202411040254	组员	前端辅助开发，参与页面功能测试、前端交互调整、检索与综述页面辅助完善	15%
郝思怡	202411040237	组员	文档润色与格式整理，参与需求分析、测试记录整理、报告内容校对和文字规范化	15%
王淑珍	202411040256	组员	PPT 制作与展示材料整理，参与系统截图收集、演示流程梳理和答辩辅助准备	15%

周思源作为组长承担项目主要开发与统筹工作，负责将需求、架构、核心功能、联调、文档和汇报串联起来。其他成员围绕后端辅助、前端辅助、文档整理和展示材料完成协作任务。小组自评比例反映了各成员在项目开发、测试、文档和展示准备中的贡献。

附录 A 代码

附录 A 补充展示部分核心代码结构，完整代码以项目仓库为准。

代码 A-1 backend-ai 文献向量化文本构建

```
def build_vector_text(doc: DocumentItem) -> str:
    parts = [
        f"标题: {doc.title}",
        f"标题: {doc.title}",
        f"学科分类: {doc.categoryName}",
        f"文献类型: {doc.documentType}",
        f"关键词: {doc.keywords}",
```

```

        f"关键词: {doc.keywords}",
        f"摘要: {doc.abstractText}",
        f"正文节选: {doc.content}",
    ]
    return "\n".join(parts)

```

代码 A-2 LLM 参考文献后端补全

```

private String ensureReferencesComplete(String content, List<Literature> literatures) {
    String standardRefs = buildStandardReferences(literatures);
    int refIndex = content.indexOf("六、参考文献来源");
    if (refIndex == -1) {
        return content + "\n\n 六、参考文献来源\n\n" + standardRefs;
    }
    int lineEnd = content.indexOf('\n', refIndex);
    if (lineEnd == -1) {
        lineEnd = refIndex + "六、参考文献来源".length();
    }
    String before = content.substring(0, lineEnd);
    return before + "\n\n" + standardRefs;
}

```

代码 A-3 非法引用编号清理

```

private String cleanInvalidCitations(String content, int maxRef)
{
    Matcher matcher = CITATION_PATTERN.matcher(content);
    StringBuffer sb = new StringBuffer();
    while (matcher.find()) {
        int num = Integer.parseInt(matcher.group(1));
        if (num < 1 || num > maxRef) {
            matcher.appendReplacement(sb, "");
        }
    }
    matcher.appendTail(sb);
    return sb.toString();
}

```

附录 B 测试报告

表 B-1 边界与异常测试数据

测试编号	输入或场景	预期结果	说明
B-001	综述生成只选择 1 篇文献	返回“请至少选择 2 篇参考文献”	验证下边界
B-002	综述生成选择 6 篇文献	返回“最多选择 5 篇参考文献”	验证上边界

		或前端阻止	
B-003	智能检索 query 为空	返回参数错误或不提交请求	验证空输入
B-004	backend-ai 未启动时重建索引	返回智能检索服务未启动提示	验证服务异常
B-005	删除或移走 FAISS 索引后检索	提示请先重建智能检索索引	验证索引缺失
B-006	active LLM 配置不存在	自动降级为 rule	generationMode=llm_fallback_rule
B-007	LLM 返回空响应	自动降级为 rule	防止页面无结果
B-008	LLM 输出缺少正文部分	自动降级为 rule	检查前五节完整性
B-009	LLM 输出缺少参考文献来源	后端补全或替换第六节	保证参考来源一致
B-010	普通用户访问管理员接口	返回 403	验证权限控制

测试结论：系统核心功能能够按照设计流程运行，普通用户端、管理员端、backend-ai 和 LLM 增强流程均有对应测试用例。实际验收时应结合浏览器截图、接口返回和终端启动状态填写实际结果。

附录 C 参考文献

- [1] Ian Sommerville. Software Engineering[M]. Pearson, 2015.
- [2] Roger S. Pressman, Bruce R. Maxim. Software Engineering: A Practitioner's Approach[M]. McGraw-Hill, 2020.
- [3] Spring Boot Reference Documentation[EB/OL]. <https://docs.spring.io/spring-boot/>
- [4] React Documentation[EB/OL]. <https://react.dev/>
- [5] Vite Documentation[EB/OL]. <https://vite.dev/>
- [6] MyBatis-Plus Documentation[EB/OL]. <https://baomidou.com/>
- [7] MySQL 8.0 Reference Manual[EB/OL]. <https://dev.mysql.com/doc/>
- [8] FastAPI Documentation[EB/OL]. <https://fastapi.tiangolo.com/>
- [9] FAISS Documentation[EB/OL]. <https://faiss.ai/>
- [10] Sentence Transformers Documentation[EB/OL]. <https://www.sbert.net/>
- [11] Patrick Lewis, Ethan Perez, Aleksandra Piktus, et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks[C]. NeurIPS, 2020.
- [12] DeepSeek API Pricing[EB/OL]. https://api-docs.deepseek.com/quick_start/pricing

附录 D AI 辅助的交互记录

本项目在需求分析、技术路线讨论、代码问题排查、测试用例设计、文档结构整理、图表设计和报告润色过程中使用了 AI 辅助。AI 主要用于提供思路建议、帮助梳理软件工程文档结构、检查文档与代码的一致性、辅助生成 Mermaid 图表和改写说明文字。项目最终的功能整合、运行测试、截图验收、代码取舍和提交判断由小组成员人工完成。

AI 辅助使用过程中遵守以下原则：不把未实现功能写成已实现；不写真实 API Key、数据库密码或 Token；不把课程项目模拟数据描述为真实论文库；不声称系统是完整工业级 RAG 平台；不直接复制超长聊天原文进入报告。AI 辅助记录作为学习和协作过程说明，不替代小组成员的项目实现与验收工作。